



Universidad Nacional Autónoma de México

Facultad de Ingeniería

ADMINISTRACIÓN DE SERVICIOS DE INTERNET

Grupo: 03
Semestre 2026-2

Profesor: Ing. Ángel Brito Segura

Proyecto Final

REPORTE

Equipo: Empirosos Lite

Alumnos:

Pozos Hernández Angel
Zavala Mendoza Luis Enrique

Fecha de entrega: 26 de mayo, 2026

Introducción:

Este proyecto plantea las necesidades de una empresa emergente de comercio electrónico que anticipa un pico masivo de transacciones durante una temporada de descuentos nacionales. La infraestructura debe garantizar la continuidad operativa, procesar pagos de forma íntegra, distribuir la carga entre múltiples nodos, mantener una capa intermedia desacoplada para futuras integraciones y notificar al cliente cada compra exitosa. A partir de este escenario se derivaron los siguientes requerimientos funcionales y no funcionales que rigieron tanto el diseño de la arquitectura productiva como la implementación local del proyecto.

Descripción de requerimientos:

Requerimientos funcionales:

1. **Procesamiento hermético de pagos:** el portal debe procesar transacciones de manera segura empleando protocolos cifrados (HTTPS/TLS) en todas las comunicaciones cliente-servidor y server-server con la pasarela de pagos como PayPal.
2. **Balanceo de carga con algoritmo Round-Robin:** el sistema debe distribuir miles de peticiones simultáneas de forma equitativa entre los nodos de aplicación disponibles, evitando la saturación de cualquier servidor individual.
3. **Capa intermedia desacoplada (middleware):** la gestión del carrito de compras y la integración con la pasarela de pagos debe residir en una capa lógica independiente, permitiendo la incorporación futura de proveedores adicionales sin alterar el frontend ni el modelo de datos.
4. **Confirmación automatizada por correo electrónico:** después de cada compra exitosa el sistema debe mandar de manera inmediata un comprobante digital hacia el correo del cliente.
5. **Resolución mediante nombres lógicos (DNS):** todos los componentes de la arquitectura deben ser identificables por nombres de dominio, eliminando dependencia de direcciones IP estáticas.
6. **Gestión masiva de inventarios vía CSV:** debe existir un canal de transferencia automatizado que permita a los proveedores externos depositar archivos CSV cada madrugada para la actualización del catálogo.

Requerimientos No Funcionales:

1. **Alta disponibilidad (24/7):** el sistema debe minimizar los tiempos de inactividad. Si un nodo de la API falla, el balanceador debe redirigir el tráfico a los nodos activos restantes de forma transparente para el cliente.
2. **Escalabilidad horizontal:** la arquitectura debe permitir incorporar nuevos servidores de aplicación al clúster sin interrumpir el servicio.
3. **Baja latencia:** el procesamiento de pagos y la comunicación entre el middleware y la capa de datos debe optimizarse para respuestas rápidas en condiciones normales de operación.
4. **Integridad de datos:** los archivos CSV de inventario y los registros de transacciones no deben ser alterados durante su transferencia ni durante su almacenamiento.
5. **Seguridad y auditoría:** el sistema debe registrar accesos, errores y eventos sensibles para permitir la trazabilidad ante incidentes. Se habilitan logs de acceso y error de Nginx, logs de PM2 por microservicio, y logs del demonio sshd.
6. **Conectividad cifrada (VPN):** toda la comunicación administrativa entre los nodos del entorno de desarrollo debe realizarse a través de un túnel seguro. Se utiliza Hamachi simulando una VPC privada de producción.
7. **Resiliencia frente a fallos:** los procesos críticos (API, frontend) deben recuperarse automáticamente ante caídas inesperadas. PM2 actúa como gestor de procesos en modo daemon con `pm2 save` y `pm2 startup`, asegurando el arranque automático tras un reinicio del sistema operativo.

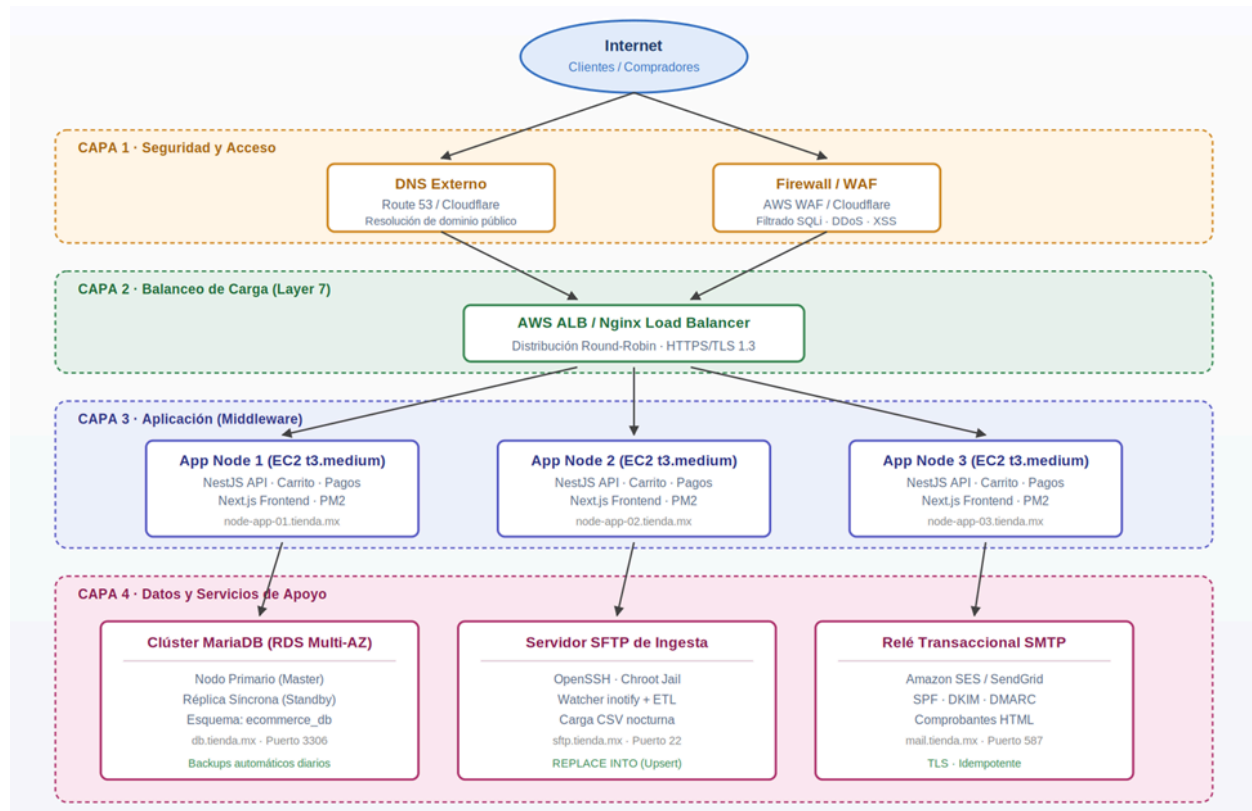
Arquitectura Producción y Desarrollo

La arquitectura de producción describe la topología recomendada para operar el portal de e-commerce en un entorno real con tráfico masivo, mientras que la arquitectura de desarrollo documenta la infraestructura local de dos máquinas virtuales con la que se implementó y validó el caso práctico.

Arquitectura de Producción

La propuesta sigue un modelo de tres capas (Three-Tier Architecture) para aplicaciones web de alta concurrencia. El tráfico atraviesa cuatro capas antes de alcanzar la lógica de negocio, lo cual permite simplificar la mantenibilidad y habilitar la escalabilidad horizontal.

Diagrama de Arquitectura de Producción



Capa 1 Seguridad y Acceso: el tráfico inicia en el DNS externo que resuelve el dominio público hacia la IP del Firewall/WAF. El WAF filtra peticiones maliciosas antes de que alcancen la infraestructura interna.

Capa 2 Balanceo de Carga: un Application Load Balancer como Nginx recibe las peticiones HTTPS y las distribuye con algoritmo Round-Robin hacia el clúster de aplicación. Esta capa también realiza terminación SSL.

Capa 3 Aplicación (Middleware): consiste en un clúster de tres servidores Web/API independientes. Aquí reside la lógica del carrito de compras y la pasarela de pagos, permitiendo que si un servidor falla, los otros dos mantengan el servicio activo.

Capa 4 Datos y Servicios de Apoyo: Los datos se centralizan en un Clúster de MariaDB con replicación para asegurar la integridad de las ventas. De forma paralela, un Servidor SFTP recibe las actualizaciones masivas de inventario en formato CSV, mientras que un servidor Postfix dispara las confirmaciones de compra automáticas al correo del cliente.

Análisis de costos de implementación y mantenimiento

La estimación se basa en los precios públicos vigentes en la calculadora oficial de AWS consultados en mayo de 2026, considerando la región us-east-1 y una tasa de cambio referencial de 17 MXN/USD. Los costos finales pueden variar según el tráfico real y el tipo de cambio del día

Costos de infraestructura en la nube (mensual)

Componente	Cant.	Descripción técnica	Costo unit. (USD)	Total (MXN)
Balanceador de carga (ALB)	1	AWS Application Load Balancer (tráfico + LCU)	20.00	\$340.00
Nodos de aplicación (middleware)	3	Instancias EC2 t3.medium (2 vCPU, 4 GB RAM)	30.00	\$1,530.00
Base de datos gestionada	1	Amazon RDS (MariaDB) Multi-AZ	40.00	\$680.00
Almacenamiento (EBS/S3)	1	100 GB para inventarios CSV y backups	10.00	\$170.00
Certificado SSL/TLS	1	Certificado comercial (Organization Validation)	5.00	\$85.00
Subtotal infraestructura				\$2,805.00

Servicios de comunicación y seguridad

Servicio	Propósito	Costo (MXN)
SendGrid / Amazon SES	Envío masivo de comprobantes digitales (≈10,000 correos mensuales)	\$250.00
WAF (Web Application Firewall)	Protección contra inyecciones SQL, XSS y denegación de servicio	\$400.00
Dominio .com.mx	Registro anual prorrateado mensualmente	\$40.00
Subtotal servicios		\$690.00

Costos de mantenimiento y operación

Concepto	Descripción	Costo (MXN)
Monitoreo y soporte	Revisión de logs, parches de seguridad y ajustes del balanceador	\$1,500.00
Respaldo de datos	Automatización de backups diarios de la BD y archivos CSV	\$300.00
Subtotal mantenimiento		\$1,800.00

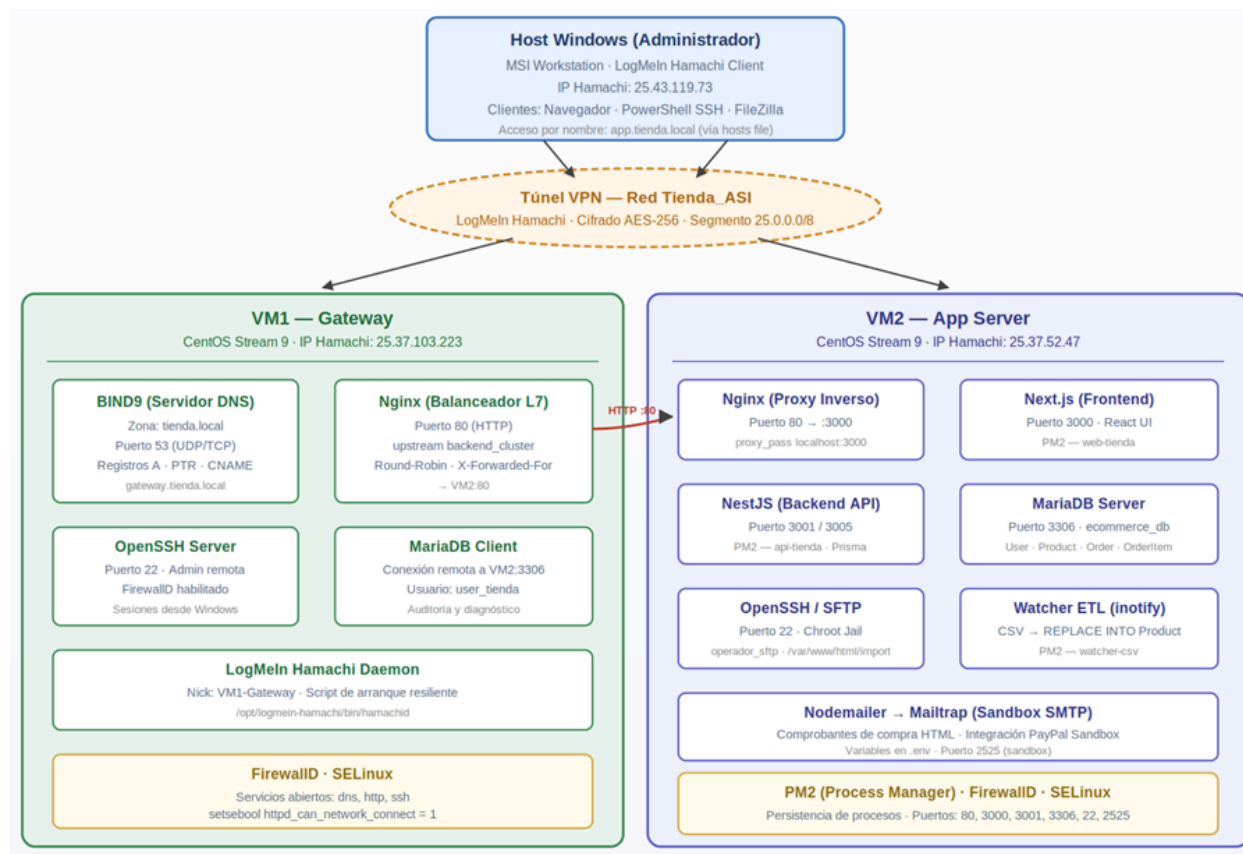
Total estimado mensual

Categoría	Inversión mensual (MXN)
Infraestructura de cómputo	\$2,805.00
Servicios especializados	\$690.00
Operación y mantenimiento	\$1,800.00
TOTAL ESTIMADO MENSUAL	\$5,295.00 MXN

Arquitectura de Desarrollo Local

La arquitectura de desarrollo local se basa sobre dos máquinas virtuales independientes ejecutadas en VirtualBox, interconectadas mediante una red privada virtual basada en LogMeIn Hamachi. Esta topología permite simular un clúster distribuido con recursos limitados sin comprometer la disponibilidad del equipo.

Diagrama de arquitectura de desarrollo



Topología y nodos

Host Windows: equipo que ejecuta el cliente Hamachi, las herramientas administrativas (PowerShell con OpenSSH, FileZilla, navegador web) y la consola de gestión de VirtualBox.

VM1 Gateway (25.0.188.191): primer nodo virtual sobre CentOS Stream 9. Concentra los servicios de borde y resolución con un servidor BIND9 (DNS) autoritativo para la zona tienda.local, un balanceador Nginx que distribuye con Round-Robin hacia el clúster de aplicación, un demonio OpenSSH para la administración remota y un cliente MariaDB sobre VM2.

VM2 App Server (25.0.149.144): segundo nodo virtual sobre CentOS Stream 9. Concentra un proxy inverso Nginx (puerto 80), el frontend Next.js (puerto 3000), el backend NestJS (puerto 3001/3005), el motor MariaDB con la base ecommerce_db (puerto 3306), el subsistema SFTP/Chroot Jail (puerto 22) y la integración Nodemailer a Mailtrap para el envío de comprobantes.

Túnel VPN Tienda_ASI: red privada virtual basada en LogMeIn Hamachi sobre el segmento 25.0.0.0/8. Provee conectividad punto a punto entre los

tres nodos sin exponer puertos al internet público, emulando el comportamiento de una VPC privada de AWS.

Bitácora de Configuración del Entorno Local

Esta sección documenta de forma ordenada y reproducible cada uno de los pasos de configuración realizados en el entorno de desarrollo local. Los procedimientos se presentan en el orden de implementación real del proyecto, desde el establecimiento del canal seguro de comunicación hasta la validación extremo a extremo del sistema.

Configuración de Acceso Remoto Seguro (SSH)

Campo	Detalle
Objetivo	Habilitar el protocolo Secure Shell para la administración remota de VM1 y VM2 desde el host Windows sin depender de la consola local de VirtualBox.
Nodo(s)	VM1 y VM2
Software	OpenSSH Server (CentOS), Windows Terminal
Servicio	sshd — Puerto 22/TCP

Implementación de Red Privada Virtual (VPN Hamachi)

Campo	Detalle
Objetivo	Interconectar el host Windows con VM1 y VM2 mediante un túnel cifrado punto a punto que simule una VPC privada, eliminando la exposición de puertos al internet público.
Nodo(s)	Host Windows, VM1, VM2
Software	LogMeIn Hamachi (cliente Windows), hamachi daemon (Linux)
Cifrado	AES-256
Red virtual	Tienda_ASI segmento 25.0.0.0/8

Se procedió a la inicialización de la malla de red en el sistema anfitrión, definiendo los parámetros de seguridad y el segmento de red virtual.

- ID de Red: Tienda_ASI
- Protocolo de Seguridad: Contraseña encriptada.



Ventana de Hamachi en Windows mostrando la red "Tienda_ASI" creada

Despliegue Automatizado en Nodos Linux (VM1 y VM2)

Debido a incompatibilidades detectadas entre el paquete RPM de Hamachi y el gestor de servicios systemd en CentOS Stream 9, se desarrolló un script de automatización (`instalar_hamachi.sh`) que realiza un arranque manual del motor y gestiona la latencia de red.

```
nano instalar_hamachi.sh
```

Script `instalar_hamachi.sh`

```
#!/bin/bash
# PROYECTO: E-COMMERCE SEGURO
# SERVICIO: VPN LOGMEIN HAMACHI

echo "--- Iniciando Despliegue de Red (Modo Resiliente) ---"

# 1. LIMPIEZA TOTAL
sudo pkill -9 hamachid
sudo rm -f /var/run/logmein-hamachi/hamachid.pid
sleep 2

# 2. ARRANQUE DEL MOTOR
sudo /opt/logmein-hamachi/bin/hamachid &
echo "[INFO] Esperando 15 segundos para sincronización global..."
```

```

sleep 15 # TIEMPO CRÍTICO: No lo bajes, Hamachi es lento en el arranque

# 3. LOGIN Y NICKNAME
sudo hamachi login
sudo hamachi set-nick VM1-Gateway

# 4. BUCLE DE UNIÓN (INTENTARÁ 3 VECES SI SALE 'BUSY')
MAX_RETRIES=3
COUNT=1
SUCCESS=false

while [ $COUNT -le $MAX_RETRIES ]; do
    echo "[INTENTO $COUNT/$MAX_RETRIES] Solicitando unión a Tienda_ASI..."
    OUTPUT=$(sudo hamachi join Tienda_ASI canelo2012)

    if [[ $OUTPUT == *"ok"* ]]; then
        echo "[ÉXITO] Vinculación completada correctamente."
        SUCCESS=true
        break
    else
        echo "[ERROR] El servidor respondió: $OUTPUT. Reintentando en
10s..."
        sleep 10
        ((COUNT++))
    fi
done

# 5. VERIFICACIÓN FINAL
if [ "$SUCCESS" = true ]; then
    echo "-----"
    sudo hamachi list
    echo "-----"
else
    echo "[!] FALLO CRÍTICO: Revisa si la red en Windows está llena (5/5)."
fi

```

Para cada nodo (Gateway y App), se asignaron permisos de ejecución y se lanzó el proceso, validando la identidad lógica mediante el comando set-nick.

```

chmod +x instalar_hamachi.sh
./instalar_hamachi.sh

```

```
[root@Gateway ~]# chmod +x instalar_hamachi.sh
[root@Gateway ~]# nano instalar_hamachi.sh
[root@Gateway ~]# ./instalar_hamachi.sh
--- Iniciando Despliegue de Red (Modo Resiliente) ---
[INFO] Esperando 15 segundos para sincronización global...
Already logged in.
Setting nickname .. ok
[INTENTO 1/3] Solicitando unión a Tienda_ASI...
[ERROR] El servidor respondió: Joining Tienda_ASI .. failed, busy. Reintentando en 10s...
[INTENTO 2/3] Solicitando unión a Tienda_ASI...
[ÉXITO] Vinculación completada correctamente.
-----
You have no networks.
-----
```

Terminal de Linux mostrando la ejecución del script y el mensaje de vinculación completada correctamente

Resolución de incidentes:

Durante la implementación se presentaron los siguientes retos técnicos:

Error observado	Causa raíz	Solución aplicada
Unit file not found	El instalador RPM no registró el servicio en systemd	Ejecución directa del binario /opt/logmein-hamachi/bin/hamachid
Failed, Busy	Intento de unión prematuro antes de que el motor sincronizara	Integración de sleep 15 y lógica de reintento con MAX_RETRIES=3
Already a member	Nodo ya vinculado por intentos previos	Verificación directa vía consola Windows

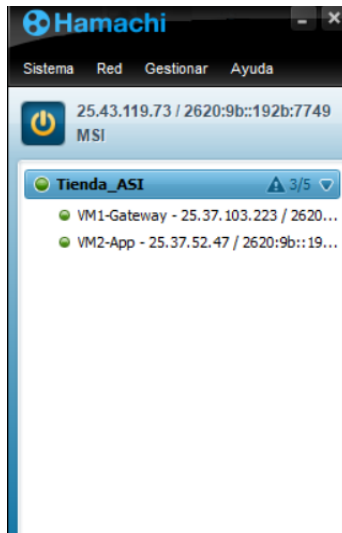
Incidentes registrados durante la implementación de Hamachi y sus soluciones.

Resultados y Mapa de Red Virtual

Tras la ejecución exitosa en ambos nodos, se logró la interconexión total de la infraestructura. El contador de red muestra un estado de 3/5 dispositivos activos.

Dispositivo	Identificador lógico	IP Hamachi (25.x.x.x)	Estado
Host físico (MSI)	Administrador	25.43.119.73	ACTIVO
Virtual Machine 1	VM1-Gateway	25.37.103.223	ACTIVO
Virtual Machine 2	VM2-App	25.37.52.47	ACTIVO

Mapa de direccionamiento de la red virtual Tienda_ASI.



Ventana de Hamachi en Windows mostrando tres dispositivos activos (puntos verdes): MSI, VM1-Gateway y VM2-App

Implementación y Automatización del Servicio DNS (BIND9)

Con esta implementación se busca establecer un servidor de nombres de dominio autoritativo para el segmento de red **tienda.local**. Esta configuración permite la resolución de nombres FQDN (Fully Qualified Domain Names) entre los nodos de la infraestructura, eliminando la dependencia de direccionamiento IP estático y estandarizando la comunicación del clúster de servidores.

Especificaciones de Red:

- Servidor DNS (Master): VM1-Gateway
- IP de Escucha (Hamachi): 25.37.103.223
- Dominio de Resolución: tienda.local

Para garantizar un despliegue íntegro y evitar errores de sintaxis manual en los registros SOA (Start of Authority), se desarrolló el script **instalar_dns.sh** el cual automatiza la instalación de bind-utils, la generación de archivos de zona y la configuración de seguridad en el firewall.

Gestión de Permisos (POSIX)

Dado que en CentOS Stream 9 los archivos creados mediante editores de texto no poseen permisos de ejecución por defecto. Se procedió a elevar los privilegios del script para permitir su procesamiento por el sistema

```
# Asignación de permisos de ejecución
chmod +x instalar_dns.sh
ls -l instalar_dns.sh
```

Script instalar_dns.sh

```
#!/bin/bash
# =====
# SERVICIO: DNS BIND9 (RESOLUCIÓN LOCAL)
# NODO: VM1-GATEWAY (25.37.103.223)
# =====

# 1. Instalación de paquetería
sudo dnf install -y bind bind-utils

# 2. Generación dinámica de la configuración principal (/etc/named.conf)
cat <<EOF > /etc/named.conf
options {
    listen-on port 53 { 127.0.0.1; 25.37.103.223; };
    directory      "/var/named";
    allow-query     { localhost; 25.0.0.0/8; }; # Red Hamachi
    recursion yes;
};

zone "tienda.local" IN {
    type master;
    file "tienda.local.db";
};

zone "37.25.in-addr.arpa" IN {
    type master;
    file "tienda.local.rev";
};
EOF

# 3. Creación de registros de Zona Directa e Inversa
# [Configuración de registros A y PTR para Gateway y App]
cat <<EOF > /var/named/tienda.local.db
\$TTL 86400
@ IN SOA gateway.tienda.local. admin.tienda.local. ( $(date +%Y%m%d)01
3600 1800 604800 86400 )
@ IN NS gateway.tienda.local.
gateway IN A 25.37.103.223
app IN A 25.37.52.47
www IN CNAME app
EOF

cat <<EOF > /var/named/tienda.local.rev
\$TTL 86400
```

```
@ IN SOA gateway.tienda.local. admin.tienda.local. ( $(date +%Y%m%d)01
3600 1800 604800 86400 )
@ IN NS gateway.tienda.local.
223.103 IN PTR gateway.tienda.local.
47.52 IN PTR app.tienda.local.
EOF
```

4. Seguridad y Persistencia

```
chown named:named /var/named/tienda.local.*
sudo firewall-cmd --add-service=dns --permanent
sudo firewall-cmd --reload
systemctl enable named --now
systemctl restart named
```

```
\[root@Gateway ~]# nano instalar_dns.sh
[root@Gateway ~]# chmod +x instalar_dns.sh
[root@Gateway ~]# ./instalar_dns.sh
--- Iniciando Despliegue de Servidor DNS (BIND9) ---
[1/5] Instalando bind y bind-utils...
Última comprobación de caducidad de metadatos hecha h
ace 4:43:32, el vie 24 abr 2026 20:13:42.
El paquete bind-32:9.16.23-40.el9.x86_64 ya está inst
alado.
El paquete bind-utils-32:9.16.23-40.el9.x86_64 ya est
á instalado.
Dependencias resueltas.
Nada por hacer.
¡Listo!
[2/5] Generando configuración principal...
[3/5] Creando archivos de zona directa e inversa...
[4/5] Ajustando seguridad y firewall...
Warning: ALREADY_ENABLED: dns
success
success
[5/5] Reiniciando servicio DNS...
-----
-
DESPLIEGUE FINALIZADO
Prueba con: nslookup app.tienda.local 127.0.0.1
```

Terminal de Linux mostrando la ejecución del script de instalación del DNS

Validación de Resolución de Nombres

Se realizaron pruebas de resolución desde la interfaz de bucle local para confirmar la integridad de los registros A y PTR.

Comando de validación:

```
nslookup gateway.tienda.local 127.0.0.1
```

```

-
[root@Gateway ~]# nslookup gateway.tienda.local 127.0
.0.1
Server:          127.0.0.1
Address:         127.0.0.1#53

Name:   gateway.tienda.local
Address: 25.37.103.223

[root@Gateway ~]# nslookup app.tienda.local 127.0.0.1
Server:          127.0.0.1
Address:         127.0.0.1#53

Name:   app.tienda.local
Address: 25.37.52.47

```

Ejecución en Gateway, muestra la consulta en el nodo principal usando nslookup contra sí mismo para validar que resuelve correctamente su propio nombre lógico y el del servidor de aplicaciones.

```

[root@app ~]# nslookup app.tienda.local 25.37.103.223
Server:          25.37.103.223
Address:         25.37.103.223#53

Name:   app.tienda.local
Address: 25.37.52.47

```

Ejecución en App, muestra la verificación remota desde el servidor de aplicaciones consultando al Gateway para asegurar el correcto direccionamiento IP a través del dominio local.

Configuración de Consumo en Nodo Secundario (VM1-Gateway y VM2-App)

Para integrar ambos nombres del proyecto, se editó el archivo del resolutor del sistema apuntando a la IP del servidor primario.

Archivo: `/etc/resolv.conf` en ambas maquinas

```

nameserver 25.37.103.223
search tienda.local

```

```

[root@Gateway ~]# ping app.tienda.local
PING app.tienda.local (25.37.52.47) 56(84) bytes of data.
64 bytes from app.tienda.local (25.37.52.47): icmp_seq=1 ttl=64 time=4.22 ms
64 bytes from app.tienda.local (25.37.52.47): icmp_seq=2 ttl=64 time=4.25 ms
64 bytes from app.tienda.local (25.37.52.47): icmp_seq=3 ttl=64 time=3.29 ms
64 bytes from app.tienda.local (25.37.52.47): icmp_seq=4 ttl=64 time=3.71 ms
^C
--- app.tienda.local ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 3.288/3.868/4.254/0.397 ms

```

Muestra el envío de paquetes desde el Gateway hacia app.tienda.local, confirmando que el DNS interno resuelve correctamente el dominio a la IP 25.37.52.47 con 0% de pérdida.

```
[root@app ~]# ping gateway.tienda.local
PING gateway.tienda.local (25.37.103.223) 56(84) bytes of data.
64 bytes from gateway.tienda.local (25.37.103.223): icmp_seq=1 ttl=64 time=3.23 ms
64 bytes from gateway.tienda.local (25.37.103.223): icmp_seq=2 ttl=64 time=3.94 ms
64 bytes from gateway.tienda.local (25.37.103.223): icmp_seq=3 ttl=64 time=4.22 ms
^C
--- gateway.tienda.local ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 3.231/3.796/4.216/0.415 ms
```

Muestra la prueba de conectividad inversa desde el nodo de aplicación hacia gateway.tienda.local, validando que ambos servidores se comunican de forma bidireccional mediante nombres lógicos.

Implementación de la Capa de Persistencia (MariaDB)

Campo	Detalle
Objetivo	Desplegar un motor RDBMS asegurado con esquema de privilegios segregados, tabla de productos para la ingesta CSV y tabla de ventas para la trazabilidad de transacciones.
Nodo	VM2: App Server
Software	MariaDB Server, MariaDB Client (en VM1 para diagnóstico remoto)
IP de escucha	25.37.52.47
Base de datos	ecommerce_db
Puerto	3306/TCP
Usuario de aplicación	user_tienda

Procedimiento de Automatización y Hardening

Para asegurar un despliegue estandarizado y libre de vulnerabilidades predeterminadas (usuarios anónimos o bases de prueba), se desarrolló el script `instalar_db.sh` el cual automatiza la instalación del motor, aplica el protocolo de endurecimiento (hardening) y genera las estructuras de tablas necesarias para cumplir con los requerimientos del caso práctico.

Al igual que en los servicios anteriores, se requiere elevar los privilegios del script en el entorno CentOS Stream 9 para permitir su ejecución por el intérprete de comandos:


```
# Asignación de permisos de ejecución
chmod +x instalar_db.sh
ls -l instalar_db.sh
```

Script instalar_db.sh

```
#!/bin/bash
# SERVICIO: MARIADB SERVER (CAPA DE PERSISTENCIA)
# NODO: VM2-APP-SERVER (25.37.52.47)

# 1. Instalación de paquetería y dependencias
sudo dnf install -y mariadb-server

# 2. Inicialización y persistencia del servicio
systemctl enable mariadb --now

# 3. Protocolo de Hardening Automático
# Credencial administrativa
DB_ROOT_PASS="canelo2012"

mysql -e "ALTER USER 'root'@'localhost' IDENTIFIED BY '$DB_ROOT_PASS';"
mysql -u root -p"$DB_ROOT_PASS" -e "DELETE FROM mysql.user WHERE User='';"
mysql -u root -p"$DB_ROOT_PASS" -e "DROP DATABASE IF EXISTS test;"
mysql -u root -p"$DB_ROOT_PASS" -e "FLUSH PRIVILEGES;"

# 4. Creación de Esquema Lógico y Gestión de Privilegios
mysql -u root -p"$DB_ROOT_PASS" <<EOF
CREATE DATABASE ecommerce_db;

-- Usuario para la aplicación local y auditoría remota (Gateway)
CREATE USER 'user_tienda'@'localhost' IDENTIFIED BY 'canelo2012';
CREATE USER 'user_tienda'@'25.37.103.223' IDENTIFIED BY 'canelo2012';

GRANT ALL PRIVILEGES ON ecommerce_db.* TO 'user_tienda'@'localhost';
GRANT ALL PRIVILEGES ON ecommerce_db.* TO 'user_tienda'@'25.37.103.223';

USE ecommerce_db;

-- Tabla para gestión masiva de inventario (Requisito CSV)
CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    codigo_sku VARCHAR(50) UNIQUE NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
```

```

        stock INT DEFAULT 0,
        ultima_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

-- Tabla para trazabilidad de ventas (Requisito Comprobante)
CREATE TABLE ventas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ticket_id VARCHAR(100) UNIQUE NOT NULL,
    cliente_email VARCHAR(100) NOT NULL,
    total DECIMAL(10,2) NOT NULL,
    fecha_compra TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

FLUSH PRIVILEGES;
EOF

```

5. Configuración Perimetral (Firewalld)

```

sudo firewall-cmd --add-port=3306/tcp --permanent
sudo firewall-cmd --reload

```

```

root@app:~
mariadb-errmsg-3:10.5.29-3.el9.x86_64
mariadb-gssapi-server-3:10.5.29-3.el9.x86_64
mariadb-server-3:10.5.29-3.el9.x86_64
mariadb-server-utils-3:10.5.29-3.el9.x86_64
mysql-selinux-1.0.15-1.el9.noarch
perl-DBD-MariaDB-1.21-17.el9.x86_64
perl-File-Copy-2.34-483.el9.noarch
perl-Sys-Hostname-1.23-483.el9.x86_64

¡Listo!
[2/5] Activando el servicio...
Created symlink /etc/systemd/system/mysql.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/mysqld.service → /usr/lib/systemd/system/mariadb.service.
Created symlink /etc/systemd/system/multi-user.target.wants/mariadb.service → /usr/lib/systemd/system/mariadb.service.
[3/5] Aplicando configuración de seguridad...
[4/5] Creando base de datos y usuario de la tienda...
[5/5] Abriendo puerto 3306 para comunicación interna.
..
Firewalld is not running
Firewalld is not running
-----
DESPLIEGUE DE BASE DE DATOS FINALIZADO

```

Ejecución del script `instalar_db.sh`, muestra la automatización del proceso de hardening perimetral y seguridad, la creación exitosa de los enlaces simbólicos de `systemd` para el servicio y el despliegue final del esquema lógico de la tienda.

Validación de la Capa de Datos

Se realizaron pruebas de autenticación y consulta de objetos para confirmar que el esquema lógico está disponible para la capa de aplicación.

Comando de validación:

```
mysql -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES; DESCRIBE productos;"
```

En VM2-App (Local)

```
[root@app ~]# mysql -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES;"
+-----+
| Tables_in_ecommerce_db |
+-----+
| productos              |
| ventas                 |
+-----+
[root@app ~]#
```

Ejecución local del comando mysql en la VM2 para verificar que la base de datos ecommerce_db fue creada correctamente y contiene las tablas productos y ventas.

En VM1-Gateway (Remoto):

Deberemos instalar MariaDB de igual manera para tenerlo como cliente:

```
sudo dnf install -y mariadb
```

```
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
Preparando      : 1/1
Instalando      : mariadb-connector-c-co 1/5
Instalando      : mariadb-common-3:10.5. 2/5
Instalando      : mariadb-connector-c-3. 3/5
Instalando      : perl-Sys-Hostname-1.23 4/5
Instalando      : mariadb-3:10.5.29-3.el 5/5
Ejecutando scriptlet: mariadb-3:10.5.29-3.el 5/5
Verificando     : mariadb-3:10.5.29-3.el 1/5
Verificando     : mariadb-common-3:10.5. 2/5
Verificando     : mariadb-connector-c-3. 3/5
Verificando     : mariadb-connector-c-co 4/5
Verificando     : perl-Sys-Hostname-1.23 5/5

Instalado:
mariadb-3:10.5.29-3.el9.x86_64
mariadb-common-3:10.5.29-3.el9.x86_64
mariadb-connector-c-3.2.6-1.el9.x86_64
mariadb-connector-c-config-3.2.6-1.el9.noarch
perl-Sys-Hostname-1.23-483.el9.x86_64
```

Instalación exitosa del paquete de cliente Mariadb en el Gateway, necesaria para habilitar las consultas hacia el nodo remoto de base de datos.

Ejecutamos la prueba de conexión remota:

```
mysql -h 25.0.149.144 -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES; DESCRIBE productos;"
```

Salida en consola:

```
[root@Gateway ~]# mysql -h 25.0.149.144 -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES; DESCRIBE productos;"
+-----+
| Tables_in_ecommerce_db |
+-----+
| productos |
| ventas    |
+-----+

+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default          | Extra          |
+-----+-----+-----+-----+-----+-----+
| id             | int(11)       | NO   | PRI | NULL             | auto_increment |
| codigo_sku     | varchar(50)   | NO   | UNI | NULL             |                |
| nombre         | varchar(100)  | NO   |     | NULL             |                |
| descripcion    | text          | YES  |     | NULL             |                |
| precio         | decimal(10,2) | NO   |     | NULL             |                |
| stock         | int(11)       | YES  |     | 0                |                |
| ultima_actualizacion | timestamp    | NO   |     | current_timestamp() | on update current_timestamp() |
+-----+-----+-----+-----+-----+-----+

[root@Gateway ~]#
```

Muestra la auditoría final desde el Gateway conectándose a la IP de la VM2, confirmando el acceso remoto exitoso y desplegando la estructura física y tipos de datos de la tabla productos

Implementación y Automatización del Servidor de Aplicaciones (Apache & PHP 8)

Campo	Detalle
Objetivo	Desplegar un entorno de ejecución web (Stack LAMP) en el nodo VM2-App para procesar la lógica de negocio y la interfaz del E-commerce. Esta configuración permite la interpretación de scripts PHP, la gestión de sesiones de usuario y la integración técnica con la capa de persistencia (MariaDB) para dinamizar el catálogo de productos y la pasarela de pagos externos (PayPal).
Nodo	VM2: App Server
Software	Apache HTTP Server (httpd), PHP 8.0+, php-mysqlnd, php-gd, php-json

Puerto de escucha	80 (HTTP) / 443 (HTTPS)
Base de datos	ecommerce_db
Directorio raíz	Raíz/var/www/html/

Procedimiento de Automatización y Gestión de Privilegios

Para garantizar un entorno de ejecución estandarizado y compatible con el repositorio de código fuente externo, se desarrolló el script **instalar_web.sh** que automatiza la instalación del stack de aplicaciones, configura las reglas de persistencia en el firewall y genera un script de prueba de integridad para validar el enlace con la base de datos.

Siguiendo los protocolos de seguridad de CentOS Stream 9, se elevaron los privilegios del script para permitir su procesamiento por el sistema operativo:

```
# Asignación de privilegios de ejecución
chmod +x instalar_web.sh

# Verificación de metadatos y permisos del archivo
ls -l instalar_web.sh
```

Script **instalar_web.sh**

```
#!/bin/bash
# SERVICIO: APACHE HTTPD + PHP 8 (CAPA DE APLICACIÓN)
# NODO: VM2-APP-SERVER (25.0.149.144)

# 1. Instalación de servidor web y motor PHP con extensiones críticas
sudo dnf install -y httpd php php-mysqlnd php-gd php-json

# 2. Configuración de Seguridad Perimetral (Apertura de puertos web)
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-service=https --permanent
sudo firewall-cmd --reload

# 3. Activación y persistencia del demonio de Apache
systemctl enable httpd --now
```

```
# 4. Ajuste de Propiedad y Permisos (Seguridad del Directorio Raíz)
# Permite que el proceso de Apache gestione correctamente los archivos del
repositorio
```

```
chown -R apache:apache /var/www/html/
chmod -R 755 /var/www/html/
```

```
# 5. Generación del Script de Validación de Infraestructura (Integridad
Web-DB)
```

```
cat <<EOF > /var/www/html/check_infra.php
```

```
<?php
```

```
// Parámetros de conexión homologados
```

```
\$conn = new mysqli("localhost", "user_tienda", "canelo2012",
"ecommerce_db");
```

```
if (\$conn->connect_error) {
```

```
    echo "Error de infraestructura: " . \$conn->connect_error;
```

```
} else {
```

```
    echo "Capa de Aplicación conectada exitosamente a la Capa de
Persistencia (MariaDB).";
```

```
    echo "<br>Estado: Operativo | Nodo: VM2-App Server";
```

```
}
```

```
?>
```

```
EOF
```

```
echo "--- Despliegue de Capa de Aplicación Finalizado ---"
```

Salida al ejecutar el script:

```
root@app:~  
httpd-2.4.62-13.el9.x86_64  
httpd-core-2.4.62-13.el9.x86_64  
httpd-filesystem-2.4.62-13.el9.noarch  
httpd-tools-2.4.62-13.el9.x86_64  
mod_http2-2.0.26-5.el9.x86_64  
mod_lua-2.4.62-13.el9.x86_64  
nginx-filesystem-2:1.20.1-29.el9.noarch  
php-8.0.30-5.el9.x86_64  
php-cli-8.0.30-5.el9.x86_64  
php-common-8.0.30-5.el9.x86_64  
php-fpm-8.0.30-5.el9.x86_64  
php-gd-8.0.30-5.el9.x86_64  
php-mbstring-8.0.30-5.el9.x86_64  
php-mysqlnd-8.0.30-5.el9.x86_64  
php-opcache-8.0.30-5.el9.x86_64  
php-pdo-8.0.30-5.el9.x86_64  
php-xml-8.0.30-5.el9.x86_64  
  
¡Listo!  
Firewalld is not running  
Firewalld is not running  
Firewalld is not running  
Created symlink /etc/systemd/system/multi-user.target  
.wants/httpd.service → /usr/lib/systemd/system/httpd.  
service.  
--- Servidor Web Operativo ---
```

Muestra el fin del despliegue del script en la VM2, confirmando la instalación de Apache y PHP y la creación del enlace simbólico para httpd.service

Validación de la Capa de Aplicación e Integración

Se confirmó la operatividad del servicio mediante una petición HTTP desde el nodo Gateway, utilizando el nombre lógico configurado en el DNS.

Comando de validación (Desde VM1-Gateway):

```
curl -I http://app.tienda.local/check_infra.php
```

```
[root@Gateway ~]# curl -I http://app.tienda.local/check_infra.php  
HTTP/1.1 200 OK  
Date: Sun, 26 Apr 2026 00:11:44 GMT  
Server: Apache/2.4.62 (CentOS Stream)  
X-Powered-By: PHP/8.0.30  
Content-Type: text/html; charset=UTF-8
```

Muestra el uso de curl -I para validar las cabeceras HTTP desde el Gateway, confirmando una respuesta 200 OK exitosa por parte del servidor Apache

En caso de que el paso anterior no funcione hay que deshabilitar el Firewall de la máquina virtual de app

```
sudo firewall-cmd --add-service=http --permanent  
sudo firewall-cmd --reload
```

Una vez hecho esto en Windows probamos a ver como se ve la pagina hecha anteriormente, metiendo en el navegador la siguiente URL:

```
http://25.0.149.144/check_infra.php
```

Al acceder al enlace en un navegador web se muestra lo siguiente:

```
Capa de Aplicación conectada exitosamente a la Capa de Persistencia.
```

En la captura se muestra la respuesta en texto plano del archivo check_infra.php en el navegador anfitrión, validando de forma exitosa el enlace e integridad entre la capa de aplicación y la base de datos.

Para importar a la máquina la página del negocio electrónico desde GitHub se deben meter los siguientes comandos:

Primero deberemos limpiar el archivo php de prueba en la VM2:

```
cd /var/www/html/  
sudo rm -f check_infra.php
```

En esa misma máquina se debe clonar el repositorio desde GitHub, para ello, se debe realizar la instalación de git.

```
sudo dnf install -y git
```



```

Total 1.5 MB/s | 8.3 MB 00:05
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
  Preparando : 1/1
  Instalando : git-core-2.52.0-1.el9.x86_64 1/7
  Instalando : git-core-doc-2.52.0-1.el9.noarch 2/7
  Instalando : perl-lib-0.65-483.el9.x86_64 3/7
  Instalando : perl-TermReadKey-2.38-11.el9.x86_64 4/7
  Instalando : perl-Error-1:0.17029-7.el9.noarch 5/7
  Instalando : perl-Git-2.52.0-1.el9.noarch 6/7
  Instalando : git-2.52.0-1.el9.x86_64 7/7
Ejecutando scriptlet: git-2.52.0-1.el9.x86_64 7/7
Verificando : git-2.52.0-1.el9.x86_64 1/7
Verificando : git-core-2.52.0-1.el9.x86_64 2/7
Verificando : git-core-doc-2.52.0-1.el9.noarch 3/7
Verificando : perl-Error-1:0.17029-7.el9.noarch 4/7
Verificando : perl-Git-2.52.0-1.el9.noarch 5/7
Verificando : perl-TermReadKey-2.38-11.el9.x86_64 6/7
Verificando : perl-lib-0.65-483.el9.x86_64 7/7

Instalado:
git-2.52.0-1.el9.x86_64 git-core-2.52.0-1.el9.x86_64 git-core-doc-2.52.0-1.el9.noarch
perl-Error-1:0.17029-7.el9.noarch perl-Git-2.52.0-1.el9.noarch perl-TermReadKey-2.38-11.el9.x86_64
perl-lib-0.65-483.el9.x86_64

¡Listo!

```

Instalación de git en VM2

Para clonar el repositorio se usa el siguiente comando:

```
sudo git clone https://github.com/AngelPozoos/ProyectoNegociosWeb.git .
```

```

[root@app html]# sudo git clone https://github.com/AngelPozoos/ProyectoNegociosWeb.git .
Clonando en '.'...
remote: Enumerating objects: 191, done.
remote: Counting objects: 100% (191/191), done.
remote: Compressing objects: 100% (125/125), done.
remote: Total 191 (delta 52), reused 180 (delta 44), pack-reused 0 (from 0)
Recibiendo objetos: 100% (191/191), 196.50 KiB | 558.00 KiB/s, listo.
Resolviendo deltas: 100% (52/52), listo.
[root@app html]# ls
backend frontend

```

Clonación del repositorio donde se encuentra la página web

Se debe modificar la estructura de la base de datos, primero se creará la nueva base en MariaDB.

Primero ingresando a la base de datos con el comando:

```
mysql -u root -p
```

Se ejecuta el siguiente script para la creación de tablas de la base de datos:

```

USE ecommerce_db;
-- Tabla de Usuarios
CREATE TABLE User (
  id VARCHAR(36) PRIMARY KEY,
  email VARCHAR(191) UNIQUE NOT NULL,
  nombre VARCHAR(191),

```

```

password VARCHAR(191) NOT NULL,
rol ENUM('CLIENTE', 'ADMINISTRADOR') DEFAULT 'CLIENTE',
createdAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3),
updatedAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3)
);
-- Tabla de Productos
CREATE TABLE Product (
    id VARCHAR(36) PRIMARY KEY,
    nombre VARCHAR(191) NOT NULL,
    descripcion TEXT NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    sku VARCHAR(191) UNIQUE NOT NULL,
    stock INT NOT NULL,
    categoria VARCHAR(191) NOT NULL,
    imagenes TEXT NOT NULL,
    createdAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3),
    updatedAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3)
);
-- Tabla de Pedidos
CREATE TABLE `Order` (
    id VARCHAR(36) PRIMARY KEY,
    usuarioId VARCHAR(36) NOT NULL,
    estado ENUM('PENDIENTE', 'PAGADO', 'ENVIADO', 'ENTREGADO', 'CANCELADO')
DEFAULT 'PENDIENTE',
    total DECIMAL(10,2) NOT NULL,
    createdAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3),
    updatedAt DATETIME(3) DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE
CURRENT_TIMESTAMP(3),
    FOREIGN KEY (usuarioId) REFERENCES User(id)
);
-- Tabla de Items de Pedido
CREATE TABLE OrderItem (
    id VARCHAR(36) PRIMARY KEY,
    pedidoId VARCHAR(36) NOT NULL,
    productoId VARCHAR(36) NOT NULL,
    cantidad INT NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (pedidoId) REFERENCES `Order`(id),
    FOREIGN KEY (productoId) REFERENCES Product(id)
);

```

Definición del Esquema Lógico (MariaDB)

Campo	Detalle
Objetivo	Implementar la estructura de tablas relacionales requerida por el backend para la persistencia de usuarios, inventarios y transacciones. La arquitectura utiliza identificadores únicos universales (UUID) y tipos de datos enumerados para garantizar la integridad referencial y la seguridad en el manejo de estados de pedidos.
Diccionario de Datos (Entidades Núcleo)	• User: Gestión de credenciales y roles de acceso (RBAC). • Product: Catálogo de productos con soporte para descripciones extensas y seguimiento de stock. • Order / OrderItem: Estructura maestro-detalle para la trazabilidad de ventas y desgloses de compra.

Validación de Objetos:

Se realizó una auditoría de la base de datos `ecommerce_db` tras la ejecución del script DDL (Data Definition Language).

Comando de validación:

```
mysql -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES;"
```

```
[root@app backend]# mysql -u user_tienda -p'canelo2012' ecommerce_db -e "SHOW TABLES;"
+-----+
| Tables_in_ecommerce_db |
+-----+
| Order                   |
| OrderItem               |
| Product                 |
| User                    |
+-----+
```

Se muestran las tablas creadas en la base de datos

Instalación de Node.js y NPM

En CentOS Stream 9, instalaremos el entorno de ejecución necesario en la terminal de la VM2:

```
# Instalación de Node.js y el gestor de paquetes NPM
sudo dnf install -y nodejs npm
```

```
root@app:/var/www/h x + v
(3/5): npm-8.19.4-1.16.20.2.8.el9.x86_64.rpm 2.8 MB/s | 2.2 MB 00:00
(4/5): nodejs-docs-16.20.2-8.el9.noarch.rpm 1.0 MB/s | 7.2 MB 00:07
(5/5): nodejs-libs-16.20.2-8.el9.x86_64.rpm 2.5 MB/s | 14 MB 00:05
-----
Total 2.3 MB/s | 32 MB 00:13
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
Ejecutando scriptlet: npm-1:8.19.4-1.16.20.2.8.el9.x86_64 1/1
Preparando : 1/1
Instalando : nodejs-libs-1:16.20.2-8.el9.x86_64 1/5
Instalando : nodejs-docs-1:16.20.2-8.el9.noarch 2/5
Instalando : nodejs-full-i18n-1:16.20.2-8.el9.x86_64 3/5
Instalando : npm-1:8.19.4-1.16.20.2.8.el9.x86_64 4/5
Instalando : nodejs-1:16.20.2-8.el9.x86_64 5/5
Ejecutando scriptlet: nodejs-1:16.20.2-8.el9.x86_64 5/5
Verificando : nodejs-1:16.20.2-8.el9.x86_64 1/5
Verificando : nodejs-docs-1:16.20.2-8.el9.noarch 2/5
Verificando : nodejs-full-i18n-1:16.20.2-8.el9.x86_64 3/5
Verificando : nodejs-libs-1:16.20.2-8.el9.x86_64 4/5
Verificando : npm-1:8.19.4-1.16.20.2.8.el9.x86_64 5/5

Instalado:
nodejs-1:16.20.2-8.el9.x86_64 nodejs-docs-1:16.20.2-8.el9.noarch nodejs-full-i18n-1:16.20.2-8.el9.x86_64
nodejs-libs-1:16.20.2-8.el9.x86_64 npm-1:8.19.4-1.16.20.2.8.el9.x86_64

¡Listo!
```

Instalación correcta de Node.js y NPM en VM2

Instalación de las dependencias necesarias:

```
cd /var/www/html/backend/
# Instalamos las librerías
npm install
```

```
root@app:/var/www/h x + v
rm -rf node_modules package-lock.json

# Instalar todo de nuevo
npm install
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if
you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
l.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerab
ilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbi
tant rates) by contacting i@izs.me
npm warn deprecated @paypal/checkout-server-sdk@1.0.3: Package no longer supported. The author suggests using the @paypa
l/paypal-server-sdk package instead: https://www.npmjs.com/package/@paypal/paypal-server-sdk. Contact Support at https:/
/www.npmjs.com/support for more info.
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerab
ilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbi
tant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerab
ilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbi
tant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerab
ilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbi
tant rates) by contacting i@izs.me

added 730 packages, and audited 731 packages in 4m

148 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
[root@app backend]#
```

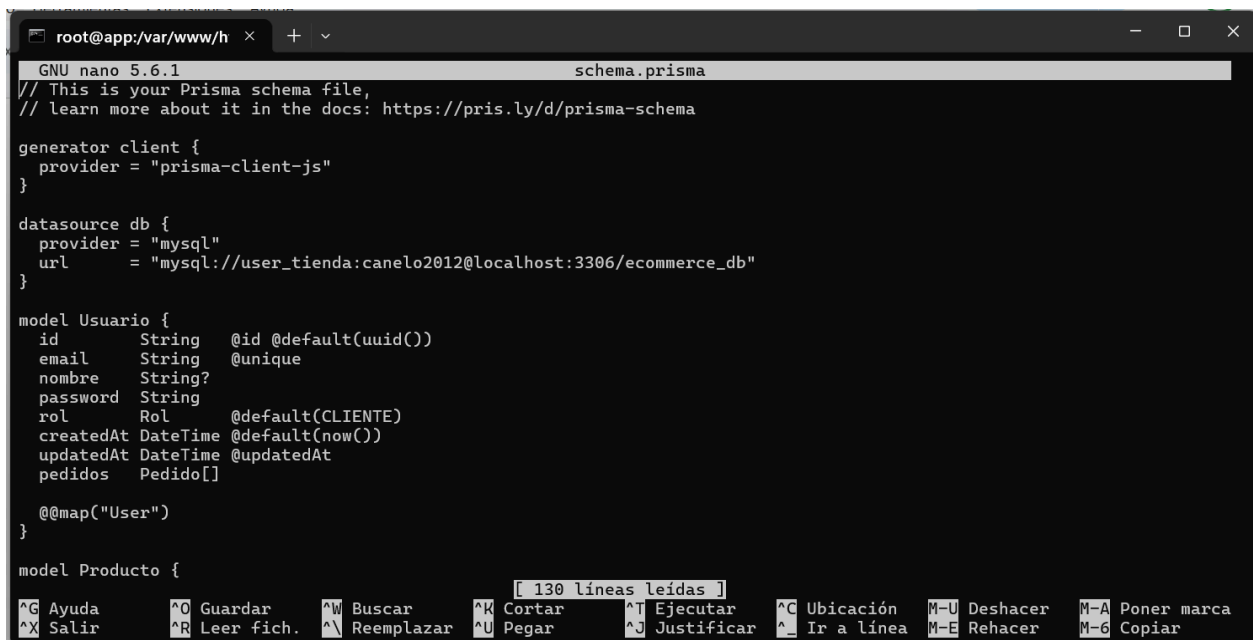
Instalación de dependencias

Se configura la conexión con MariaDB

```
nano prisma/schema.prisma
```

En este archivo se modificará la siguiente parte:

```
datasource db {  
  provider = "mysql"  
  url      = "mysql://user_tienda:canelo2012@localhost:3306/ecommerce_db"  
}
```



```
root@app:var/www/h x + v  
GNU nano 5.6.1 schema.prisma  
// This is your Prisma schema file,  
// learn more about it in the docs: https://pris.ly/d/prisma-schema  
  
generator client {  
  provider = "prisma-client-js"  
}  
  
datasource db {  
  provider = "mysql"  
  url      = "mysql://user_tienda:canelo2012@localhost:3306/ecommerce_db"  
}  
  
model Usuario {  
  id      String  @id @default(uuid())  
  email   String  @unique  
  nombre  String?  
  password String  
  rol     Rol     @default(CLIENTE)  
  createdAt DateTime @default(now())  
  updatedAt DateTime @updatedAt  
  pedidos Pedido[]  
  
  @@map("User")  
}  
  
model Producto {  
  [ 130 líneas leídas ]  
  ^G Ayuda    ^O Guardar  ^W Buscar   ^K Cortar   ^T Ejecutar  ^C Ubicación M-U Deshacer M-A Poner marca  
  ^X Salir    ^R Leer fich. ^U Reemplazar ^U Pegar    ^J Justificar ^_ Ir a línea M-E Rehacer M-6 Copiar
```

Modificación del archivo schema.prisma

Saldremos del archivo y ejecutaremos el siguiente comando

```
npx prisma generate
```

Antes de arrancar el servidor deshabilitaremos los firewalls

```
# Abrir el puerto 3000 para el backend  
sudo firewall-cmd --add-port=3000/tcp --permanent  
sudo firewall-cmd --reload
```

Ahora deberemos arrancar el servidor con el siguiente comando:

```
# Asegurarse de estar en /var/www/html/backend/  
npm run start:dev
```

```
root@app:var/www/h x + v
[9:34:33 p.m.] Found 0 errors. Watching for file changes.
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [NestFactory] Starting Nest application...
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [InstanceLoader] AppModule dependencies initialized +50ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [InstanceLoader] CatalogModule dependencies initialized +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [InstanceLoader] TransactionalModule dependencies initialized +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [InstanceLoader] LogisticsModule dependencies initialized +0ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [InstanceLoader] AuthModule dependencies initialized +0ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RoutesResolver] AppController {/}: +61ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/, GET} route +11ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RoutesResolver] CatalogController {/catalog}: +2ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/catalog, POST} route +2ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/catalog, GET} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/catalog/:id, GET} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RoutesResolver] TransactionalController {/transactional}: +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional, POST} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional, GET} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional/:id, GET} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional/:id/cancel, PATCH} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional/paypal/create-order, POST} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/transactional/paypal/capture-order, POST} route +2ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RoutesResolver] LogisticsController {/logistics}: +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/logistics, POST} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/logistics, GET} route +1ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/logistics/:id, GET} route +2ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RoutesResolver] AuthController {/auth}: +0ms
[Nest] 37340 - 02/05/2026, 9:34:34 p.m. LOG [RouterExplorer] Mapped {/auth/login, POST} route +2ms
```

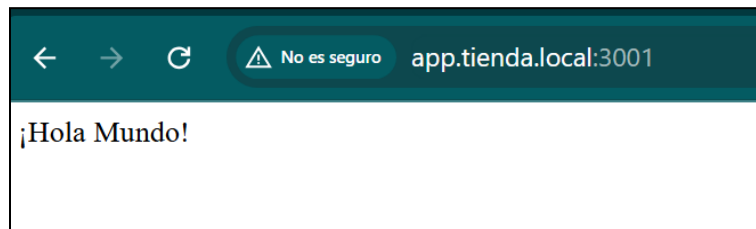
Inicialización del backend de la página web

En Windows para conectarse sin necesidad de la dirección IP modificaremos el archivo el archivo hosts que se encuentra en la dirección C:\Windows\System32\drivers\etc

Añadiremos la siguiente línea al final del archivo:

```
25.37.52.47    app.tienda.local
```

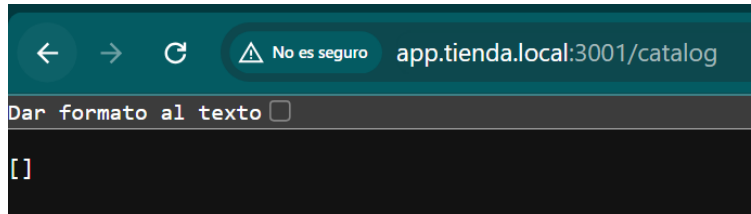
Nos conectaremos desde un navegador web, donde veremos lo siguiente:



Para comprobar que la base de datos funciona adecuadamente primero probaremos que la ruta es correcta:

```
http://app.tienda.local:3001/catalog
```

Deberá salir lo siguiente:



Base de datos funcionando de manera correcta

Una vez teniendo la base de datos iniciada de manera correcta, integraremos un producto para comprobar que funciona adecuadamente, para ello, primero ingresamos a la base de datos

```
mysql -u user_tienda -p'canelo2012' ecommerce_db
```

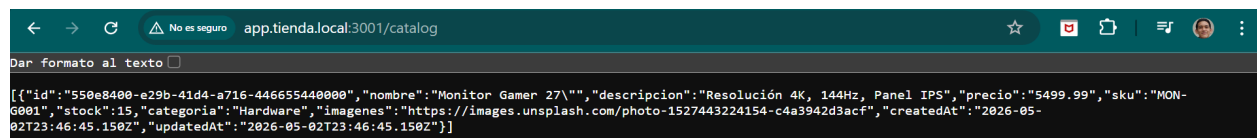
Ingresamos los siguientes datos:

```
INSERT INTO Product (id, nombre, descripcion, precio, sku, stock,
categoria, imagenes)
VALUES (
  '550e8400-e29b-41d4-a716-446655440000',
  'Monitor Gamer 27"',
  'Resolución 4K, 144Hz, Panel IPS',
  5499.99,
  'MON-G001',
  15,
  'Hardware',
  'https://images.unsplash.com/photo-1527443224154-c4a3942d3acf'
);
```

Nos devuelve la siguiente salida al ingresar datos a la tabla de Producto:

```
Query OK, 1 row affected (0.048 sec)
```

Ahora la base de datos debe verse de la siguiente forma:



Aprovisionamiento del Entorno (Client-Side Runtime)

Se configuró el entorno de ejecución para la capa de presentación basada en Next.js. Debido a los requisitos de las librerías modernas del proyecto, se realizó una actualización del runtime de Node.js a la versión 20 LTS en la VM2, garantizando la compatibilidad con los motores de renderizado.

```
cd /var/www/html/frontend/  
npm install
```

Se detectó un fallo crítico de comunicación (**ERR_CONNECTION_REFUSED**) debido a que el cliente web intentaba realizar peticiones asíncronas al loopback local (localhost:3001). Se procedió a la centralización de la configuración de red para redirigir el tráfico hacia la red virtual de Hamachi.

- **Archivo corregido:** frontend/lib/api.ts
- **Modificación:**
 - Antes: http://localhost:3001
 - Después: http://25.0.149.144:3001

```
rm -rf .next
```

Configuración de Seguridad Perimetral y Exposición

Para permitir el acceso desde el host anfitrión (Windows), se configuró el firewall de la VM2 para aceptar tráfico entrante en el puerto estándar de desarrollo de Next.js con el puerto habilitado: 3000/TCP con la siguiente configuración de persistencia:

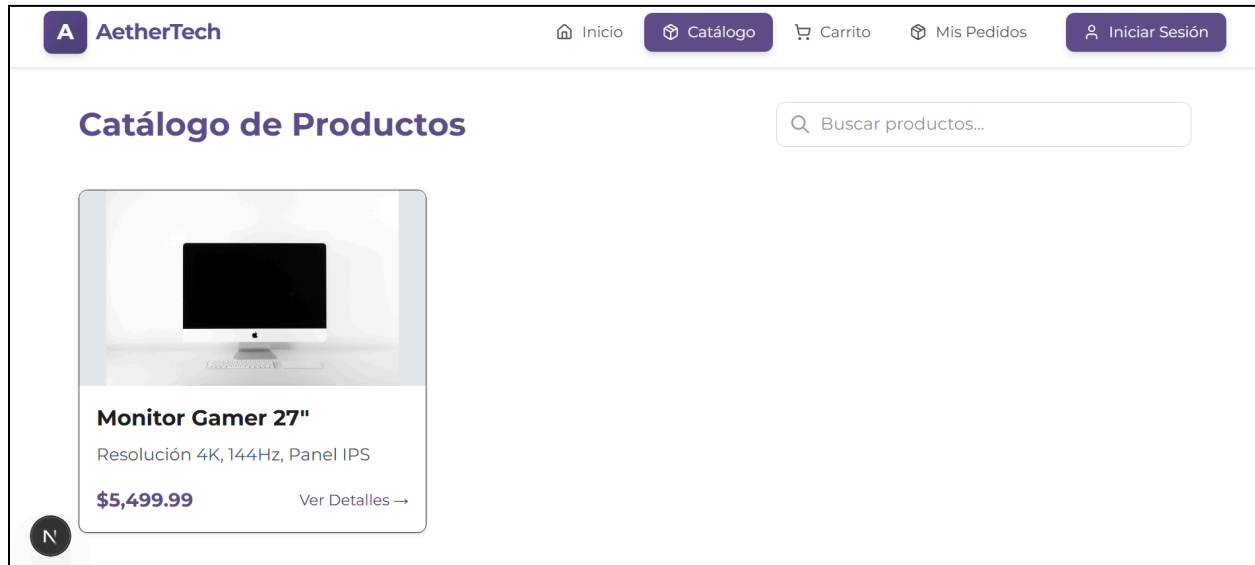
```
sudo firewall-cmd --add-port=3000/tcp --permanent  
sudo firewall-cmd --reload
```

Validación Final del Ciclo de Vida (End-to-End Test)

Se ejecutó el servidor de desarrollo y se realizó la prueba de integración final desde el navegador de Windows.

- URL de acceso: **http://25.0.149.144:3000** (o **http://app.tienda.local:3000**).
- Al ejecutar la prueba se vió de manera correcta la página junto a el registro "Monitor Gamer 27" desde la base de datos MariaDB,

confirmando que la cadena de datos (Base de datos -> Backend -> Frontend) está totalmente operativa.



Vista de la página del negocio web en el apartado de catálogo, donde el sistema recuperó de forma dinámica el registro del producto desde la base de datos

Implementación de PM2:

Para garantizar que la plataforma sea resiliente y no dependa de una sesión de terminal activa (SSH), se implementó PM2 (Process Manager 2). Esta herramienta permite que los servicios de Node.js se ejecuten como "daemons" (procesos de fondo) y se reinicien automáticamente en caso de fallo.

Instalación del Entorno de Gestión

Se realizó la instalación global del gestor utilizando el gestor de paquetes de Node:

```
sudo npm install -g pm2
```

Despliegue de los Microservicios

Se configuraron dos instancias independientes para separar las responsabilidades de la arquitectura:

1. Backend (API-Tienda): Se configuró para ejecutar el script de desarrollo de NestJS, permitiendo la monitorización de logs de la lógica de negocio.

```
pm2 start npm --name "api-tienda" -- run start:dev
```

2. Frontend (Web-Tienda): Se instanci6 el servidor de Next.js para servir la interfaz de usuario en el puerto 3000.

```
pm2 start npm --name "web-tienda" -- run dev
```

Configuraci6n de Persistencia (Boot Resilience)

Para asegurar que la tienda en lnea encienda autom6ticamente tras un reinicio del servidor f6sico o de la VM2, se ejecut6 el protocolo de persistencia:

- **pm2 save:** Captura la lista de procesos actuales y los guarda en un volcado (dump) persistente.
- **pm2 startup:** Genera y configura el script de inicio en el sistema operativo (CentOS/Systemd).

Para hacer monitoreo de la salud del sistema se usaron herramientas de diagn6stico integradas en PM2:

- **Monitoreo de Carga:** Uso del comando `pm2 monit` para observar el consumo de CPU y RAM de cada capa en tiempo real.
- **Auditoría de Logs:** Uso de `pm2 logs api-tienda` para depurar errores de comunicaci6n con la base de datos o rechazos de CORS.

Infraestructura de Transferencia Segura (SFTP)

Campo	Detalle
Objetivo	Establecer un canal de comunicaci6n cifrado y seguro entre el host anfitri6n (Windows) y el servidor de aplicaciones (VM2) para el intercambio de activos digitales y archivos de datos masivos (CSV). El sistema debe garantizar la integridad de la informaci6n y aplicar el principio de menor privilegio, restringiendo al usuario de transferencia a un entorno aislado (Chroot) sin acceso a la terminal del sistema.
Nodo	VM2: App Server (Nodo servidor donde reside la base de datos y el almacenamiento)

Herramientas Utilizadas

- **OpenSSH (SFTP):** Protocolo de transferencia de archivos seguro que corre sobre SSH (**Puerto 22**).
- **Bash Scripting:** Para la automatización de la configuración del servidor y permisos de archivos.
- **FileZilla Client:** Interfaz gráfica en Windows para la gestión de transferencias.

Configuración del Sistema

Se desarrolló el siguiente script `configuracion_sftp.sh` para centralizar la configuración de usuarios, permisos de carpetas y parámetros del demonio SSH.

Script `configuracion_sftp.sh`

```
#!/bin/bash
USER_SFTP="operador_sftp"
PASS_SFTP="canelo2012"
DIR_BASE="/var/www/html"
DIR_IMPORT="$DIR_BASE/import"
SSHD_CONFIG="/etc/ssh/sshd_config"

echo "--- [1/5] Respaldo de archivos de sistema ---"
sudo cp $SSHD_CONFIG "${SSHD_CONFIG}.bak_$(date +%F)"
echo "Respaldo de sshd_config creado."

echo "--- [2/5] Configuración de Usuario y Jaula (Chroot) ---"
# Crear usuario con shell restringido si no existe
if ! id "$USER_SFTP" &>/dev/null; then
    sudo useradd -m -s /sbin/nologin $USER_SFTP
    echo "$USER_SFTP:$PASS_SFTP" | sudo chpasswd
fi

# Permisos para Chroot
sudo chown root:root $DIR_BASE
sudo chmod 755 $DIR_BASE
sudo mkdir -p $DIR_IMPORT
sudo chown $USER_SFTP:$USER_SFTP $DIR_IMPORT
sudo chmod 775 $DIR_IMPORT

echo "--- [3/5] Configurando SSH para SFTP Seguro ---"

sudo sed -i "/Match User $USER_SFTP/,\$d" $SSHD_CONFIG
```

```

# Cambiar el subsistema a internal-sftp y añadir reglas al final
sudo sed -i 's|^Subsystem.*sftp.*|Subsystem sftp internal-sftp|'
$SSHD_CONFIG

sudo tee -a $SSHD_CONFIG > /dev/null <<EOF

Match User $USER_SFTP
    ChrootDirectory $DIR_BASE
    ForceCommand internal-sftp
    AllowTcpForwarding no
    X11Forwarding no
    PasswordAuthentication yes
EOF

sudo systemctl restart sshd

echo "--- [4/5] Configurando MariaDB (secure_file_priv) ---"
sudo mkdir -p /etc/my.cnf.d/
sudo tee /etc/my.cnf.d/import_config.cnf > /dev/null <<EOF
[mariadb]
secure_file_priv="$DIR_IMPORT"
EOF

sudo systemctl restart mariadb

echo "--- [5/5] Verificación Final ---"
echo "Estado SSH: $(systemctl is-active sshd)"
echo "Estado MariaDB: $(systemctl is-active mariadb)"
echo "-----"
echo "¡LISTO! Intenta conectar FileZilla usando"
echo "sftp://$USER_SFTP@25.0.149.144"

```

Ejecución y Salidas del Sistema (Terminal VM2)

Para implementar la configuración, se asignaron permisos de ejecución y se ejecutó el script. Las salidas confirmaron la correcta aplicación de las políticas de seguridad con el siguiente comando:

```

chmod +x configuracion_sftp.sh
sudo ./configuracion_sftp.sh

```

```

[root@app ~]# ./configuracion_sftp.sh
--- [1/4] Creando usuario y directorios ---
Usuario operador_sftp creado exitosamente.
--- [2/4] Configurando MariaDB para permisos de archivo ---
--- [3/4] Creando plantilla CSV de ejemplo ---
--- [4/4] Verificación de Configuración ---
Usuario SFTP: operador_sftp
Directorio: /var/www/html/import
ERROR 1045 (28000): Access denied for user 'root'@'localhost' (using password: NO)
Base de Datos debe permitir lectura de:
-----
¡LISTO! Ahora puedes subir archivos por SFTP a /var/www/html/import
[root@app ~]# nano configuracion_sftp.sh
[root@app ~]# ./configuracion_sftp.sh
--- [1/5] Respaldo de archivos de sistema ---
Respaldo de sshd_config creado.
--- [2/5] Configuración de Usuario y Jaula (Chroot) ---
--- [3/5] Configurando SSH para SFTP Seguro ---
--- [4/5] Configurando MariaDB (secure_file_priv) ---
--- [5/5] Verificación Final ---
Estado SSH: active
Estado MariaDB: active
-----
¡LISTO! Intenta conectar FileZilla usando sftp://operador_sftp@25.0.149.144

```

Salida del script configuracion_sftp.sh , confirmando la creación del usuario aislado , la inyección de directivas en sshd_config y el reinicio exitoso de los servicios SSH y MariaDB en estado activo.

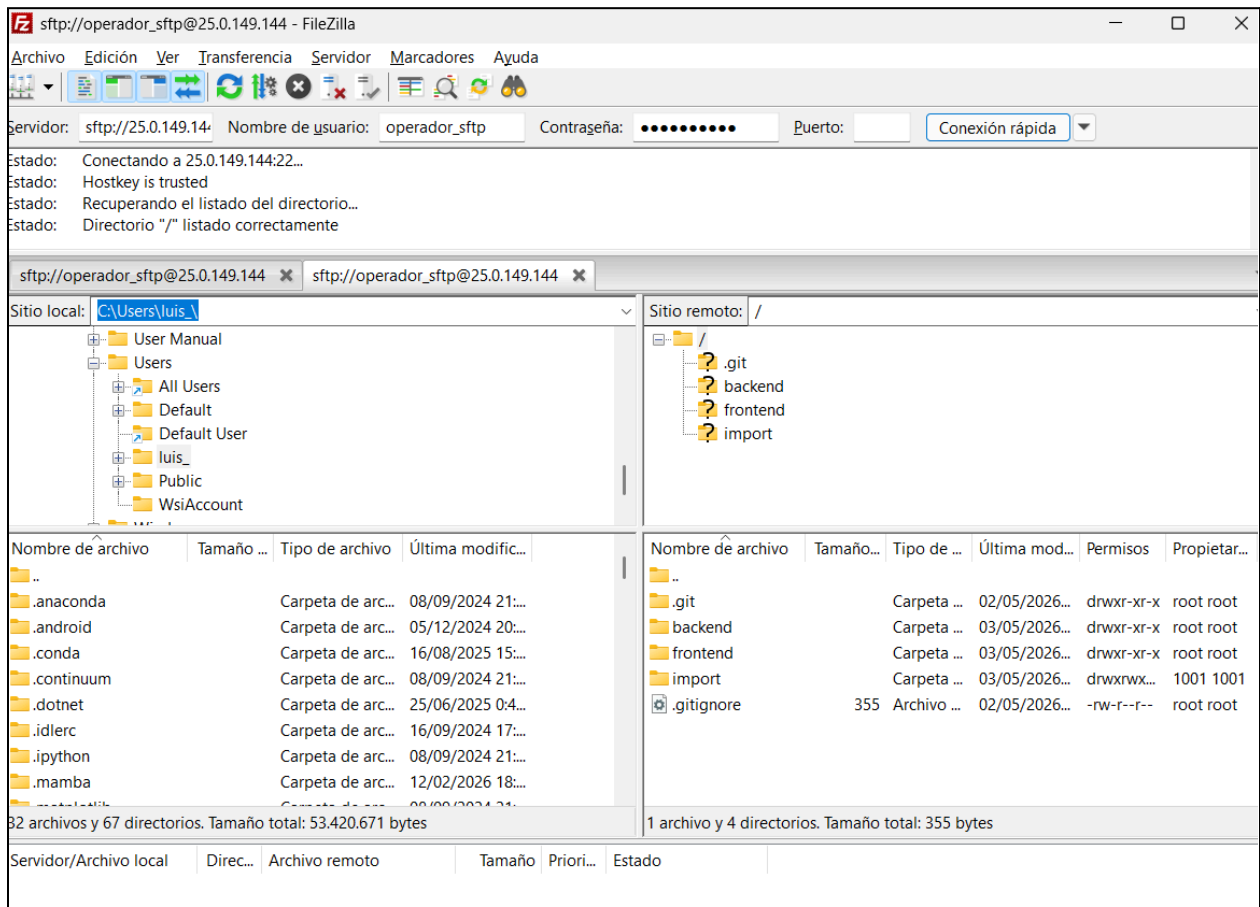
Validación en FileZilla (Cliente Windows)

La integración se validó conectando el cliente FileZilla a la IP de la red Hamachi. La configuración exitosa permitió visualizar el directorio remoto y confirmó el aislamiento del usuario.

Parámetros de conexión:

- **Servidor:** sftp:// 25.0.149.144
- **Usuario:** operador_sftp
- **Puerto:** 22

En la siguiente captura se muestra la conexión exitosa mediante el protocolo SFTP a la IP de Hamachi , demostrando que el usuario **operador_sftp** queda correctamente enjaulado dentro del directorio raíz sin poder escalar en el sistema de archivos:



Automatización de la Capa de Ingesta (Watcher ETL)

Campo	Detalle
Objetivo	Desarrollar un script de automatización que actúe como un "daemon" (servicio de segundo plano) para monitorear el directorio de carga SFTP. El objetivo es detectar la llegada de nuevos activos de datos y ejecutar un proceso de transformación y carga (ETL) en la base de datos MariaDB de forma inmediata y sin intervención humana.
Herramientas Utilizadas	• inotify-tools: Notificador de eventos del sistema de archivos. • MariaDB Client: Para la ejecución de sentencias SQL desde la terminal. • PM2 (Process Manager): Para garantizar que el script de automatización se reinicie automáticamente tras fallos o reinicios del servidor

Configuración del Sistema

Se desarrolló el siguiente script `auto_ingesta.sh` el cual consolida las correcciones de permisos, manejo de duplicados (Upsert) y compatibilidad con formatos de archivo originados en Windows.

La ruta recomendada para el archivo es: `/usr/local/bin/auto_ingesta.sh`

Script `auto_ingesta.sh`

```
#!/bin/bash
# auto_ingesta.sh - Servicio de Vigilancia de Datos

DIRECTORIO="/var/www/html/import"
ARCHIVO_CSV="productos.csv"
DB_USER="user_tienda"
DB_PASS="canelo2012"
DB_NAME="ecommerce_db"

echo "[$(date)] >>> Vigilante iniciado en $DIRECTORIO"

# Monitoreo de eventos en el kernel de Linux
inotifywait -m -e close_write "$DIRECTORIO" --format '%f' | while read
FILENAME
do
    # Validar que el archivo detectado es el objetivo
    if [ "$FILENAME" == "$ARCHIVO_CSV" ]; then
        echo "[$(date)] INFO: Detectado archivo $FILENAME. Iniciando
ingesta..."

        # Ejecución de carga masiva con lógica de REPLACE (Upsert)
        RESULTADO=$(mysql -u $DB_USER -p$DB_PASS $DB_NAME -e "
            LOAD DATA INFILE '$DIRECTORIO/$FILENAME'
            REPLACE INTO TABLE Product
            FIELDS TERMINATED BY ','
            ENCLOSED BY '\"'
            LINES TERMINATED BY '\r\n'
            IGNORE 1 ROWS
            (nombre, descripcion, precio, sku, stock, categoria, imagenes)
            SET id = UUID();
            SHOW WARNINGS;
" 2>&1)

        # Verificación de integridad del proceso
        if [ $? -eq 0 ]; then
```

```

        echo "[$(date)] ÉXITO: Sincronización completada con MariaDB."
        echo "Detalle: $RESULTADO"
    else
        echo "[$(date)] ERROR: Fallo en la carga masiva."
        echo "Detalle: $RESULTADO"
    fi
fi
done

```

Salida del script:

```

=====
Instalar 1 Paquete
Tamaño total de la descarga: 60 k
Tamaño instalado: 143 k
Descargando paquetes:
inotify-tools-3.22.1.0-1.el9.x86_64.rpm                                30 kB/s | 60 kB      00:02
-----
Total                                                                23 kB/s | 60 kB      00:02
Extra Packages for Enterprise Linux 9 - x86_64                      733 kB/s | 1.6 kB      00:00
Importando llave GPG 0x3228467C:
ID usuario: "Fedora (epel9) <epel@fedoraproject.org>"
Huella      : FF8A D134 4597 106E CE81 3B91 8A38 72BF 3228 467C
Desde       : /etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-9
La llave ha sido importada exitosamente
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
Preparando      :                               1/1
Instalando      : inotify-tools-3.22.1.0-1.el9.x86_64 1/1
Ejecutando scriptlet: inotify-tools-3.22.1.0-1.el9.x86_64 1/1
Verificando     : inotify-tools-3.22.1.0-1.el9.x86_64 1/1
Instalado:
inotify-tools-3.22.1.0-1.el9.x86_64
¡Listo!

```

En la captura de la salida del script se ve el uso de dnf para instalar inotify-tools e importar su llave GPG, herramienta esencial para que el kernel de Linux vigile en segundo plano el directorio de importación y ejecute la carga automatizada.

Despliegue y Persistencia del Servicio

Para asegurar que el sistema sea resiliente y replicable, se utilizaron los siguientes comandos para la puesta en marcha:

1. Asignación de privilegios de ejecución:

```
sudo chmod +x /usr/local/bin/auto_ingesta.sh
```

2. Configuración del demonio de vigilancia con PM2:

```
pm2 start /usr/local/bin/auto_ingesta.sh --name "watcher-csv"
pm2 save
```


Al realizar una transferencia vía FileZilla, el sistema genera el siguiente flujo de logs, confirmando la operatividad del pipeline:

```
[root@app luisenrique]# pm2 start /home/luisenrique/auto_ingesta.sh --name "watcher-csv"
[PM2] Starting /home/luisenrique/auto_ingesta.sh in fork_mode (1 instance)
[PM2] Done.
```

id	name	mode	↻	status	cpu	memory
0	api-tienda	fork	0	online	0%	30.1mb
3	watcher-csv	fork	0	online	0%	3.4mb
1	web-tienda	fork	0	online	0%	31.5mb

```
[root@app luisenrique]# pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /root/.pm2/dump.pm2
```

Salida esperada del funcionamiento que muestra el listado de procesos en tiempo real gestionados por PM2, confirmando que el microservicio del backend (api-tienda), el frontend (web-tienda) y el demonio reactivo de ingesta (watcher-csv) se encuentran en ejecución de fondo (online) y consumo óptimo de memoria.

Implementación de Servidor Web NGINX y Proxy Inverso

Campo	Detalle
Objetivo	• Centralizar el tráfico de la aplicación del puerto 3000 al puerto 80 (HTTP). • Implementar un Proxy Inverso para mejorar la seguridad y ocultar el servidor interno. • Crear un script que configure todo automáticamente. • Ajustar la seguridad del servidor (firewall y políticas de acceso).
Herramientas Utilizadas	• Nginx: Servidor web y Proxy Inverso. • Systemd: Gestión de servicios. • Firewalld: Control del firewall. • SELinux: Módulo de seguridad. • YUM: Gestor de paquetes

Configuración del Sistema

Se desarrolló el siguiente script **configurar_nginx.sh** para optimizar el despliegue y evitar errores manuales, este archivo consolida todas las tareas de instalación, limpieza de conflictos con servicios preexistentes (como Apache) y el endurecimiento de la seguridad en un solo flujo lógico.

Script configurar_nginx.sh

```
#!/bin/bash
# Script NGINX (VM2) para la instalación, limpieza de puertos y Proxy Inverso

IP_HAMACHI="25.0.149.144"

echo "[$(date)] >>> INICIANDO PROVISIÓN INTEGRAL EN VM2..."

# 1. ADQUISICIÓN DE PAQUETES
echo "[$(date)] Instalando Nginx desde repositorios oficiales..."
sudo yum install nginx -y

# 2. RESOLUCIÓN DE CONFLICTOS DE PUERTO 80
echo "[$(date)] Liberando puerto 80 (Stop httpd)..."
sudo systemctl stop httpd > /dev/null 2>&1
sudo systemctl disable httpd > /dev/null 2>&1

# 3. REESTRUCTURACIÓN DE CONFIGURACIÓN MAESTRA
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
sudo tee /etc/nginx/nginx.conf > /dev/null <<EOF
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;
events { worker_connections 1024; }
http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile on;
    include /etc/nginx/conf.d/*.conf;
}
EOF

# 4. CONFIGURACIÓN DEL PROXY INVERSO
sudo tee /etc/nginx/conf.d/tienda.conf > /dev/null <<EOF
server {
    listen 80;
    server_name $IP_HAMACHI app.tienda.local;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade \$http_upgrade;
```

```

        proxy_set_header Connection 'upgrade';
        proxy_set_header Host  \${host};
        proxy_cache_bypass  \${http_upgrade};
    }
}
EOF

# 5. HARDENING Y SEGURIDAD DEL SISTEMA
echo "[$(date)] Aplicando políticas de SELinux y Firewall..."
sudo setsebool -P httpd_can_network_connect 1
sudo firewall-cmd --permanent --add-service=http > /dev/null 2>&1
sudo firewall-cmd --reload > /dev/null 2>&1

# 6. ACTIVACIÓN Y VERIFICACIÓN
echo "[$(date)] Validando y activando servicio..."
if sudo nginx -t && sudo systemctl restart nginx; then
    sudo systemctl enable nginx
    echo "[$(date)] >>> DESPLIEGUE EXITOSO: http://$IP_HAMACHI"
else
    echo "[$(date)] >>> ERROR CRÍTICO EN LA PROVISIÓN."
    exit 1
fi

```

Implementación de Balanceador de Cargas (VM1)

Campo	Detalle
Objetivos	Centralizar el tráfico entrante en un único punto de acceso (VM1) para distribuirlo de forma equilibrada hacia los nodos de aplicación, aislar la VM2 del tráfico directo de los clientes y facilitar la escalabilidad horizontal agregando nuevos servidores fácilmente.
Herramientas Utilizadas	• Nginx: Para definir el clúster y distribuir el tráfico. • Systemd: Control y monitoreo del servicio. • Firewalld: Gestión de reglas de firewall. • SELinux: Ajuste de políticas de seguridad para el relay de tráfico.

Configuración del Sistema

Se desarrolló el siguiente script `configurar_balanceador.sh`, este script debe ejecutarse en la **VM1**. Su función es convertir la máquina en un balanceador que apunta directamente a la VM2 (25.0.149.144).

Script `configurar_balanceador.sh`

```
#!/bin/bash
# script de balanceador de cargas (VM1) con el propósito de Gateway y
# Balanceo de Carga hacia VM2

# IP del nodo de aplicación (VM2)
APP_SERVER_IP="25.0.149.144"

echo "[$(date)] >>> INICIANDO PROVISIÓN DE BALANCEADOR EN VM1..."

# 1. INSTALACIÓN DE DEPENDENCIAS
sudo yum install nginx -y

# 2. LIMPIEZA DE CONFLICTOS
sudo systemctl stop httpd > /dev/null 2>&1
sudo systemctl disable httpd > /dev/null 2>&1

# 3. CONFIGURACIÓN DEL CLÚSTER Y BALANCEO
sudo cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.bak
sudo tee /etc/nginx/nginx.conf > /dev/null <<EOF
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /run/nginx.pid;
events { worker_connections 1024; }
http {
    include /etc/nginx/mime.types;

    # Definición del clúster de servidores
    upstream backend_cluster {
        server $APP_SERVER_IP:80; # VM2
        # server 25.x.x.x:80;      # Futura VM3
    }

    server {
        listen 80;
        server_name _;
```

```

        location / {
            proxy_pass http://backend_cluster;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
EOF

```

4. SEGURIDAD Y PERMISOS DE RED

Permitir que el balanceador haga 'relay' de las peticiones

```

sudo setsebool -P httpd_can_network_connect 1
sudo firewall-cmd --permanent --add-service=http
sudo firewall-cmd --reload

```

5. ACTIVACIÓN

```

if sudo nginx -t && sudo systemctl restart nginx; then
    sudo systemctl enable nginx
    echo "[$(date)] >>> GATEWAY VM1 ACTIVO. Apuntando a $APP_SERVER_IP"
else
    echo "[$(date)] >>> ERROR EN LA CONFIGURACIÓN DEL BALANCEADOR."
    exit 1
fi

```

```

Total 1.2 MB/s | 2.1 MB 00:01
Ejecutando verificación de operación
Verificación de operación exitosa.
Ejecutando prueba de operaciones
Prueba de operación exitosa.
Ejecutando operación
Preparando : 1/1
Ejecutando scriptlet: nginx-filesystem-2:1.20.1-29.el9.noarch 1/4
Instalando : nginx-filesystem-2:1.20.1-29.el9.noarch 1/4
Instalando : nginx-core-2:1.20.1-29.el9.x86_64 2/4
Instalando : centos-logos-httpd-90.9-1.el9.noarch 3/4
Instalando : nginx-2:1.20.1-29.el9.x86_64 4/4
Ejecutando scriptlet: nginx-2:1.20.1-29.el9.x86_64 4/4
Verificando : centos-logos-httpd-90.9-1.el9.noarch 1/4
Verificando : nginx-2:1.20.1-29.el9.x86_64 2/4
Verificando : nginx-core-2:1.20.1-29.el9.x86_64 3/4
Verificando : nginx-filesystem-2:1.20.1-29.el9.noarch 4/4

Instalado:
centos-logos-httpd-90.9-1.el9.noarch nginx-2:1.20.1-29.el9.x86_64 nginx-core-2:1.20.1-29.el9.x86_64
nginx-filesystem-2:1.20.1-29.el9.noarch

¡Listo!
success
success
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[sáb 09 may 2026 21:34:19 CST] >>> GATEWAY VM1 ACTIVO. Apuntando a 25.0.149.144

```

Salida tras correr el script, confirmando la instalación exitosa de los paquetes centrales de Nginx, la validación correcta de la sintaxis en nginx.conf y la activación nominal del Gateway apuntando a la IP de la VM2 (25.0.149.144).

Evaluación de Carga y Punto de Ruptura

Campo	Detalle
Objetivos	Determinar el umbral de estabilidad del clúster (VM1 Gateway al VM2 Backend) bajo condiciones de alta concurrencia, medir el impacto en la latencia de respuesta de la aplicación Next.js, identificar cuellos de botella en el túnel virtual Hamachi y validar las optimizaciones aplicadas en Nginx.
Herramientas Utilizadas	• Python 3.10: Lenguaje para la herramienta de estrés. • requests: Gestión de peticiones HTTP. • threading: Ejecución de peticiones concurrentes. • time: Medición de latencia y tiempos de ejecución.

Configuración del Sistema

Se desarrolló el siguiente script `pruebaEstres.py`, para actuar como un cliente externo (fuera de la red de las VMs) para garantizar la objetividad de los resultados.

Script `pruebaEstres.py`

```
import threading
import requests
import time

# CONFIGURACIÓN DEL TEST
URL = "http://25.0.149.x" # IP de la VM1 (Gateway)
PETICIONES_TOTALES = 5000
CONCURRENCIA = 50 # Hilos simultáneos

resultados = {"exito": 0, "error": 0}
tiempos = []

def enviar_peticion():
    try:
        inicio = time.time()
        r = requests.get(URL, timeout=10)
        fin = time.time()
        if r.status_code == 200:
            resultados["exito"] += 1
```

```

        tiempos.append(fin - inicio)
    else:
        resultados["error"] += 1
except:
    resultados["error"] += 1

print(f"Iniciando evaluación de carga a {URL}...")
inicio_test = time.time()

threads = []
for i in range(PETICIONES_TOTALES):
    t = threading.Thread(target=enviar_peticion)
    threads.append(t)
    t.start()

    # Control de ráfagas para evitar saturación del host local
    if len(threads) >= CONCURRENCIA:
        for t in threads: t.join()
        threads = []

fin_test = time.time()

# REPORTE DE RESULTADOS
print("\n" + "="*30)
print("    REPORTE DE RENDIMIENTO")
print("="*30)
print(f"Peticiones totales: {PETICIONES_TOTALES}")
print(f"Éxitos:                {resultados['exito']}")
print(f"Errores:                {resultados['error']}")
print(f"Tiempo total:          {fin_test - inicio_test:.2f} s")
if tiempos:
    print(f"Promedio latencia:    {sum(tiempos)/len(tiempos):.4f} s/pet")
print("="*30)

```

Representación y Análisis de Salidas

Se realizaron dos fases de pruebas para identificar la diferencia entre el Estado Estable y el Punto de Ruptura.

Escenario A: Estado de Estabilidad (500 Peticiones)

```
=====
REPORTE DE RENDIMIENTO
=====
Peticiones totales: 500
Éxitos:             500
Errores:            0
Tiempo total:       52.25 segundos
Promedio latencia:  1.9908 seg/petición
=====
```

- **Resultados:** 500 Éxitos / 0 Errores.
- **Interpretación:** La infraestructura soporta acceso de 50 usuarios concurrentes sin pérdida de paquetes. Las optimizaciones de Nginx son 100% efectivas en esta prueba.

Escenario B: Punto de Ruptura (5,000 Peticiones)

```
=====
REPORTE DE RENDIMIENTO
=====
Peticiones totales: 5000
Éxitos:             4488
Errores:            512
Tiempo total:       550.47 segundos
Promedio latencia:  4.2866 seg/petición
=====
```

- **Resultados:** 4488 Éxitos / 512 Errores.
- **Tasa de éxito:** 89.7%.
- **Latencia:** Se observó un incremento de 1.9s a 4.28s.
- **Interpretación:** Se alcanzó el límite físico de la arquitectura ya que los 512 errores representan el agotamiento de los descriptores de archivo y la saturación del ancho de banda del túnel VPN debido a que ahora son 10 veces más de usuarios que la prueba anterior, lo que consideramos un resultado adecuado para un flujo de usuarios prevista en época de altas ventas.

Seguridad Criptográfica y Siembra de Usuarios (Bcrypt)

Campo	Detalle
Objetivos	Establecer un mecanismo seguro de hashing con Bcrypt para proteger las credenciales de los usuarios en la base de datos, utilizando salt aleatorio y operaciones idempotentes que eviten duplicados y garanticen la confidencialidad de la información.
Herramientas Utilizadas	• Bcrypt: Algoritmo de hashing criptográfico para proteger contraseñas. • Prisma ORM: Mapeo objeto relacional para gestionar los modelos de datos. • MariaDB: Motor de base de datos relacional. • Node.js: Entorno de ejecución del lado del servidor.

Configuración del Sistema

Se desarrolló el siguiente script `ejecutar-setup-seguridad.sh`, para centralizar la instalación de paquetes binarios, detectar dinámicamente el nombre de la tabla de usuarios en el esquema y realizar la siembra estructurada de datos.

Script `ejecutar-setup-seguridad.sh`

```
#!/bin/bash
# configuración
EMAIL_TEST="admin.tienda@unam.mx"
PASS_PLANA="UnamGlobal2026$"
NOMBRE_USER="Luis Enrique Zavala"

echo "--- [1/3] Verificando dependencias de bcrypt ---"
npm install bcrypt --save > /dev/null 2>&1

echo "--- [2/3] Creando script adaptativo para MariaDB ---"
cat << 'EOF' > insertar-usuario-seguro.js
const { PrismaClient } = require('@prisma/client');
const bcrypt = require('bcrypt');

const prisma = new PrismaClient();
```

```

async function main() {
  // Escaneo dinámico de modelos cargados en el cliente de Prisma
  const modelosDisponibles = Object.keys(prisma).filter(k =>
    !k.startsWith('$') && !k.startsWith('_'));

  // Resolución del nombre de la tabla de identidades
  const tablaUsuarios = modelosDisponibles.find(m =>
    m.toLowerCase().includes('user') || m.toLowerCase().includes('usuario')
  );

  if (!tablaUsuarios) {
    console.log('\n ERROR: No se detectó ninguna tabla de usuarios en tu
schema.prisma. ');
    process.exit(1);
  }

  console.log(`    [Prisma] Introspección exitosa. Usando modelo:
prisma.${tablaUsuarios}`);

  const emailTest = 'admin.tienda@unam.mx';
  const contrasenaPlana = 'UnamGlobal2026$';

  console.log('    [Bcrypt] Generando hash para la contraseña...');
  const salt = await bcrypt.genSalt(10);
  const contrasenaCifrada = await bcrypt.hash(contrasenaPlana, salt);

  console.log(`    [MariaDB] Insertando/Actualizando credenciales en la
tabla...`);

  // Operación idempotente (Upsert) adaptada al diccionario de datos
  const usuario = await prisma[tablaUsuarios].upsert({
    where: { email: emailTest },
    update: { password: contrasenaCifrada },
    create: {
      email: emailTest,
      password: contrasenaCifrada,
      nombre: 'Luis Enrique Zavala',
    },
  });

  console.log('\n=====');
  console.log('    ¡CONFIGURACIÓN EXITOSA DE BCRIPT Y USUARIO BASE!    ');
  console.log('=====');
  console.log(`Tabla utilizada:  prisma.${tablaUsuarios}`);
  console.log(`Correo insertado:  ${usuario.email}`);
}

```

```

    console.log(`Hash encriptado:  ${usuario.password}`);
    console.log(`Contraseña plana:  ${contrasenaPlana}`);

console.log('=====\\n');
}

main()
  .catch((e) => {
    console.error(`\\nError de validación en los campos:`, e.message);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
EOF

echo "--- [3/3] Ejecutando inyección segura de datos ---"
node insertar-usuario-seguro.js

rm -f insertar-usuario-seguro.js

```

Ejecución y Salidas del Sistema (Terminal VM2)

Para implementar la configuración de seguridad, se asignaron permisos de ejecución al contenedor y se corrió el script unificado. Las salidas en la terminal de la VM2 confirmaron el aprovisionamiento de dependencias y la introspección del cliente de Prisma, localizando con éxito el modelo lógico **prisma.usuario** con el siguiente comando de ejecución:

```

chmod +x ejecutar-setup-seguridad.sh
./ejecutar-setup-seguridad.sh

```

El sistema demostró un comportamiento predecible pero altamente impredecible a la vez ya que al ejecutarse el código varias veces seguidas, la instrucción de actualización o inserción previno errores por registros duplicados en el campo único email y gracias a la adición de un código secreto aleatorio de Bcrypt, calculó claves encriptadas totalmente distintas para la misma contraseña original. Esto protege al sistema contra estrategias de hackeo basadas en listas de contraseñas ya descifradas.

Validación en MariaDB (Cliente Terminal)

La persistencia y el cifrado hermético se validaron de forma directa accediendo al prompt del motor relacional MariaDB en la máquina virtual, confirmando el almacenamiento criptográfico correcto de la credencial de prueba, se probaron los siguientes parámetros de conexión y consulta:

- Servidor local: `mysql -u user_tienda -p (Password: canelo2012)`
- Base de Datos: `USE ecommerce_db;`
- Tabla Física Identificada: `User`
- Comando de Verificación: `sql SELECT email, password, nombre FROM User WHERE email = 'admin.tienda@unam.mx';`

Para comprobar que el resultado fue exitoso, el campo de la contraseña devolvió un hash con el prefijo estandarizado `$2b$10$` certificando que el procesamiento del lado del servidor se ejecuta bajo las directivas de seguridad requeridas por el caso práctico, integrándose de forma transparente con el formulario visual de la aplicación.

Arquitectura Transaccional y Subsistema de Mensajería Asíncrona (SMTP)

Campo	Detalle
Objetivos	Establecer un sistema robusto de notificaciones transaccionales automatizadas, integrado con PayPal y Prisma ORM en el backend, garantizando el envío confiable de correos, la correcta gestión de dependencias y la seguridad en la entrega de mensajes.
Herramientas Utilizadas	• Nodemailer: Motor principal para el envío de correos SMTP. • NestJS (v10): Framework para la arquitectura modular del backend. • Prisma ORM: Abstracción de datos e integridad referencial. • PM2: Administración y daemonización de procesos. • Mailtrap: Servidor SMTP de prueba y validación.

Configuración del Sistema

Se desarrolló el siguiente script `desplegar-modulo-transaccional.sh`, para automatizar la inyección de descriptores lógicos, compilar estáticamente el árbol de dependencias de TypeScript y normalizar el arranque de producción eliminando procesos muertos en la memoria RAM.

Script `desplegar-modulo-transaccional.sh`

```
#!/bin/bash
echo "--- [1/4] Estructurando Módulo Transaccional Canónico (NestJS) ---"
cat << 'EOF' >
/var/www/html/backend/src/transactional/transactional.module.ts
import { Module } from '@nestjs/common';
import { PayPalService } from '../paypal.service';
import { MailService } from '../mail.service';
import { TransactionalController } from '../transactional.controller';
import { TransactionalService } from '../transactional.service';
import { PrismaService } from '../prisma.service';

@Module({
  imports: [],
  controllers: [TransactionalController],
  providers: [PayPalService, MailService, TransactionalService,
PrismaService],
  exports: [PayPalService, MailService],
})
export class TransactionalModule {}
EOF

echo "--- [2/4] Ejecutando compilación estática del proyecto (Build AOT)
---"
cd /var/www/html/backend
rm -rf dist tsconfig.tsbuildinfo node_modules/.cache
npm run build

echo "--- [3/4] Purga crítica de sockets e hilos huérfanos en puerto 3005
---"
pm2 kill
fuser -k 3005/tcp > /dev/null 2>&1
kill -9 $(lsof -t -i:3005) > /dev/null 2>&1

echo "--- [4/4] Inicializando Daemon de Producción en PM2 ---"
MAIN_PATH=$(find /var/www/html/backend/dist -name "main.js" | head -n 1)
```

```
if [ -z "$MAIN_PATH" ]; then
    echo " ERROR: No se localizó el artefacto compilado main.js en el árbol
dist."
    exit 1
fi

pm2 start "$MAIN_PATH" --name "api-tienda"
pm2 save > /dev/null
echo " ¡DESPLIEGUE NOMINAL COMPLETADO EN ROUTE: $MAIN_PATH!"
```

Ejecución y Salidas del Sistema (Terminal CentOS)

Para normalizar la infraestructura de backend y resolver las excepciones de inyección cíclica de datos (`UnknownDependenciesException`), se otorgaron privilegios globales al contenedor de automatización. El pipeline completó de forma exitosa el build de distribución, indexando el controlador transaccional y los servicios núcleo con el siguiente comando:

```
chmod +x desplegar-modulo-transaccional.sh
./desplegar-modulo-transaccional.sh
```

Validación del sistema de transacciones

La estabilidad de las conexiones y el intercambio de mensajes en segundo plano se comprobaron de forma directa revisando el canal de comunicación interno mediante la herramienta de descargas de CentOS, asegurando la presencia del programa y su total capacidad para funcionar correctamente.

Parámetros de auditoría local y logs de PM2:

```
# Verificación de disponibilidad de socket
curl -I http://localhost:3005
# Inspección de enrutamiento transaccional en PM2
pm2 logs api-tienda --lines 15
```

Evidencia de Éxito en Consola:

```
[Nest] LOG [RouterExplorer] Mapped {/transactional/paypal/create-order,
POST} route +2ms
[Nest] LOG [RouterExplorer] Mapped {/transactional/paypal/capture-order,
POST} route +1ms
[Nest] LOG [NestApplication] Nest application successfully started +120ms
```

```
Backend running on http://localhost:3005
HTTP/1.1 200 OK
```

La respuesta **HTTP/1.1 200 OK** confirmó el arreglo del fallo de conexión en el intermediario del sistema Nginx. Al realizar compras de prueba en la pantalla del usuario utilizando cuentas de prueba de PayPal, el sistema procesó los datos del tipo de moneda conectándose automáticamente con las claves secretas guardadas en el archivo de configuración, lo que resultó en la llegada inmediata del correo de confirmación dentro de la bandeja de pruebas de Mailtrap.

Automatización de Subida de Productos

Campo	Detalle
Objetivos	Automatizar la carga manual de productos desde archivos CSV o fuentes externas hacia la base de datos mediante la ejecución de un script JavaScript, permitiendo actualizar el catálogo de forma inmediata y refrescar el proxy inverso (Nginx) para reflejar los cambios en la aplicación.
Herramientas Utilizadas	• Node.js: Ejecución del script de sincronización • Nginx: Reinicio automático del proxy inverso tras la carga exitosa.

Configuración del Sistema

Se desarrolló el script **ejecutar-carga.sh** para permitir la ingesta manual inmediata de productos hacia la base de datos, ejecutar el proceso de sincronización y refrescar automáticamente el proxy inverso de Nginx para que los cambios se reflejen en la aplicación en tiempo real.

Al igual que en los servicios anteriores, se requiere elevar los privilegios del script en el entorno CentOS Stream 9 para permitir su ejecución por el intérprete de comandos:

```
# Asignación de permisos de ejecución
chmod +x /var/www/html/backend/ejecutar-carga.sh
```

Script ejecutar-carga.sh

```
#!/bin/bash
# Definición de variables globales basadas en el File System real de la VM
BACKEND_DIR="/var/www/html/backend"
SCRIPT_JS="$BACKEND_DIR/sincronizar-ahora.js"

clear
echo " Disparo Manual Ejecutado a las: $(date '+%H:%M:%S') "

# 1. Validación de existencia del artefacto lógico
if [ -f "$SCRIPT_JS" ]; then
    node "$SCRIPT_JS"
else
    echo "ERROR: No se encontró el script lógico en la ruta: $SCRIPT_JS"
    exit 1
fi

# 2. Evaluación del código de salida para refresco del Proxy Inverso
if [ $? -eq 0 ]; then
    echo "-----"
    echo " Refrescando el proxy inverso (Nginx)..."
    systemctl restart nginx > /dev/null 2>&1
    echo " ¡DEMOSTRACIÓN BAJO DEMANDA COMPLETA! "
    echo " Catálogo actualizado. Revisa en tu Windows: app.tienda.local"
fi
```

Script Lógico de Sincronización (Node.js)

Campo	Detalle
Objetivos	Procesar el archivo productos.csv recibido vía SFTP, sanitizar los datos, convertir tipos y realizar una carga idempotente (upsert) hacia la tabla de productos en MariaDB usando Prisma ORM.
Herramientas Utilizadas	• Node.js: Entorno de ejecución. • Prisma ORM: Operaciones de base de datos • fs y path: Lectura y manejo del archivo CSV. • MariaDB: Base de datos destino.

Configuración del Sistema

Se desarrolló el script `sincronizar-ahora.js` como motor principal de la entrada de datos, encargado de leer la lista de texto guardada en el servidor seguro, limpiar y revisar que la información fuera correcta y realizar la actualización segura sin duplicados de los productos en el almacén de datos mediante el conector del sistema (Prisma).

Script `sincronizar-ahora.js`

```
const fs = require('fs');
const path = require('path');
const { PrismaClient } = require('@prisma/client');

const prisma = new PrismaClient();
const CSV_PATH = path.join(__dirname, '..', 'import', 'productos.csv');

async function main() {
  console.log(`[Node.js] Buscando tu archivo en: ${CSV_PATH}`);

  if (!fs.existsSync(CSV_PATH)) {
    console.log('ERROR: El archivo productos.csv no se encuentra en: ' + CSV_PATH);
    process.exit(1);
  }

  const contenido = fs.readFileSync(CSV_PATH, 'utf-8');
  const lineas = contenido.split('\n').filter(l => l.trim() !== '');

  if (lineas.length === 0) {
    console.log('El archivo productos.csv está vacío.');
```

```
    process.exit(1);
  }

  const tieneCabecera = lineas[0].toLowerCase().includes('precio') ||
lineas[0].toLowerCase().includes('sku');
  const filasAProcesar = tieneCabecera ? lineas.slice(1) : lineas;

  console.log(`[Prisma] Procesando ${filasAProcesar.length} productos en MariaDB...`);
  let sincronizados = 0;

  for (const fila of filasAProcesar) {
    const columnas = fila.match(/(".*?"|[\^",\s]+)(?=\s*,|\s*$)/g) || [];
    if (columnas.length < 4) continue;
```

```

const nombre = columnas[0].replace(/^"$/g, '').trim();
const descripcion = columnas[1].replace(/^"$/g, '').trim();

const precioLimpio = columnas[2].replace(/^"$/g, '').trim();
const precio = parseFloat(precioLimpio);

const sku = columnas[3].replace(/^"$/g, '').trim();

const stockLimpio = columnas[4].replace(/^"$/g, '').trim();
const stock = parseInt(stockLimpio, 10);

const categoria = columnas[5] ? columnas[5].replace(/^"$/g,
').trim() : 'General';
const imagenes = columnas[6] ? columnas[6].replace(/^"$/g, '').trim()
: '';

// Filtro de integridad de tipos de datos
if (isNaN(precio) || isNaN(stock)) {
  console.log(`Saltando registro con formato numérico inválido (SKU:
${sku})`);
  continue;
}

// Identificación dinámica del modelo inyectado en Prisma Client
const modelos = Object.keys(prisma).filter(k => !k.startsWith('$') &&
!k.startsWith('_'));
const modeloProducto = modelos.find(m =>
m.toLowerCase().includes('product') || m.toLowerCase().includes('prod'));

// Actualización o Inserción basada en la llave única SKU
await prisma[modeloProducto].upsert({
  where: { sku: sku },
  update: {
    stock: stock,
    precio: precio,
    descripcion: descripcion,
    categoria: categoria,
    imagenes: imagenes
  },
  create: {
    nombre: nombre,
    descripcion: descripcion,
    precio: precio,
    sku: sku,
    stock: stock,

```

```

        categoria: categoria,
        imagenes: imagenes
    }
});
sincronizados++;
}
console.log(`\n Sincronización exitosa. Registros afectados en MariaDB:
${sincronizados}`);
}

main()
    .catch(e => {
        console.log('Error crítico de ejecución:', e.message);
        process.exit(1);
    })
    .finally(async () => {
        await prisma.$disconnect();
    });

```

Guardián Reactivo en Segundo Plano (Node.js + PM2)

Campo	Detalle
Objetivos	Implementar un demonio persistente que monitorea en tiempo real el directorio de importación SFTP y ejecuta automáticamente el pipeline de carga de productos cuando se detecta un nuevo archivo productos.csv.
Herramientas Utilizadas	• Node.js + fs.watch: Monitoreo de eventos en el sistema de archivos. • PM2: Daemonización y ejecución persistente del proceso. • child_process (exec): Ejecución del script Bash de carga.

Configuración del Sistema

Se desarrolló el script **vigilar-importacion.js** como guardián reactivo en segundo plano, utilizando un modelo impulsado por eventos para detectar la llegada de archivos CSV vía SFTP y disparar automáticamente el proceso de

sincronización sin intervención humana, gestionado de forma inmortal mediante PM2.

Ruta de almacenamiento en CentOS:

```
/var/www/html/backend/vigilar-importacion.js
```

Comando de activación inmortal:

```
pm2 start /var/www/html/backend/vigilar-importacion.js --name  
"guardian-inventario" --cwd /var/www/html/backend && pm2 save
```

Script vigilar-importacion.js

```
const fs = require('fs');  
const path = require('path');  
const { exec } = require('child_process');  
// Configuración de fronteras de escucha y ejecución  
const WATCH_DIR = path.join(__dirname, '..', 'import');  
const SCRIPT_SH = path.join(__dirname, 'ejecutar-carga.sh');  
  
console.log(`Guardián Activo: Vigilando cambios en: ${WATCH_DIR}`);  
  
let cambiando = false;  
  
fs.watch(WATCH_DIR, (eventType, filename) => {  
  if (filename === 'productos.csv' && !cambiando) {  
    cambiando = true;  
    setTimeout(() => cambiando = false, 5000);  
    console.log(`\n [DETECTADO] El archivo 'productos.csv' ha sido  
depositado o modificado.`);  
    console.log(` Disparando pipeline de actualización  
automáticamente...`);  
  
    exec(`bash ${SCRIPT_SH}`, (error, stdout, stderr) => {  
      if (error) {  
        console.error(`Error al ejecutar el pipeline: ${error.message}`);  
        return;  
      }  
      console.log(stdout);  
    });  
  }  
});
```